

MISRA C:2012 严格系统中 C 语言使用指南

2013 年 3 月

huanghao14

前言

在网上搜了很久，都没有 MISRA C 的中文版，故用了大力气翻译本文。

使用 MISRA C，有一句话很核心：MISRA C 是在标准 C 的基础上定义了一套 C 的规则，有一些特别的条款要只有符合 MISRA C 的编译器才能实现，因此使用不符合 MISRA C 的编译器的项目，应有选择的执行 MISRA C 规则。

一些无意义的部分没有翻译，本文件只译了文档主体内容和相关个人认为有用的附录。

因水平有限，若发现错误，请联系 amwtf@126.com。

若觉得本文还不错，又愿意感谢译者的，请微信扫一扫。



译者：黄浩

2021.1.13

1 版本说明

略

2 MISRA C 的背景

2.1 C 的流行

略

2.2 C 的缺点

C 在以下方面的存在缺陷。

2.2.1 语言定义

标准 C 未完整定义所有规则，但实现平台和实际中却运用了。

有以下几种情形：

- 行为未定义；
- 行为非预期；
- 根据设计文档自由选择应用实现平台；

依赖于未定义行为或非预期行为的程序无法保证可预期的结果。

过分依赖于实现平台的程序可能很难移植，同样可能很难运用静态分析工具进行分析。

2.2.2 语言误用

虽然 C 程序可以以结构化和可理解的方式进行编码，但 C 使程序员很容易编写晦涩难懂的代码，这使得编译器很难发现程序错误。例如下例：

```
if ( a == b )    /* tests whether a and b are equal          */
if ( a = b )    /* assigns b to a and tests whether a is non-zero */
```

2.2.3 语言误解

C 的某些领域可能让程序员较难弄懂。例如 C 语言比其他语言有更多的操作符，因此有大量不同的操作符优先级，有些操作符优先级是不直观的。

C 提供的类型规则也会让熟悉强类型语言的程序员感到困惑。例如，操作数可能被“提升”为更广泛的类型，这意味着运算产生的类型不一定与操作数的类型相同。

2.2.4 运行时错误检查

C 程序可以被编译成小而高效的机器代码，但代价是运行时检查非常有限。C 程序通常不提供对常见问题的运行时检查，比如异常算术运算（如除 0 运算）、溢出、指针的有效性、数组边界错误等。C

设计的原理是要求程序员对这些问题进行显式检查。

3 工具选择

3.1 C 及编译器

略

3.2 分析工具

略

4 预备知识

4.1 培训

为了确保 C 代码开发人员的技能和能力达到适当的水平，应为其提供以下方面培训：

- 使用 C 语言编写嵌入式应用程序
- 使用 C 语言编程高完整性和安全性相关的系统

由于编译器和静态分析工具是复杂的软件，应该考虑提供使用它们的培训。

4.2 理解编译器

编译器可以提供各种控制其行为的选项，了解这些选项的效果是很重要的，因为它们的使用或不使用可能会影响以下方面：

- 语言扩展的可用性；
- 编译器是否符合 ISO C 标准；
- 程序所需的资源，特别是处理时间和内存空间；
- 编译器中暴露缺陷的可能性，例如在执行复杂的高度优化代码转换时可能出现的缺陷。

理解编译器实现那些在 ISO C 标准中被称为“实现定义”的 C 语言的特性是很重要的。理解编译器可能提供的任何语言扩展也很重要。

编译器开发人员可能会维护一个已知的会影响编译器的缺陷列表，以及任何可用的解决方案。在使用该编译器开始项目之前，了解该列表的内容显然是有利的。如果编译器开发人员不能提供这样的列表，那么当发现缺陷或可疑缺陷并报告给编译器开发人员时，就应该维护一个本地列表。

4.3 理解静态分析工具

项目中使用的静态分析工具的文档都应该被审查，以便理解以下方面：

- 如何配置分析器来匹配编译器的实现定义的行为，例如整数类型的大小；
- 能检查哪些 MISRA C 规则条款；

- 是否能检查所有可能用到的表达式；
- 能否评估分析工具的行为，以便平衡分析时间和分析精度。

工具开发人员可能会维护一个已知的影响工具的缺陷列表，以及任何可用的解决方案。在使用这个工具开始一个项目之前，了解这个列表的内容显然是有利的。如果这样的列表不能从工具开发人员处获得，那么当发现缺陷或可疑缺陷并报告给工具开发人员时，就应该维护一个本地列表。

5 使用 MISRA C

5.1 采用 MISRA C

MISRA C 应在项目一开始就获得采用。

如果一个项目是建立在现有代码的基础上的，并且这些代码具有经过验证的跟踪记录，那么为使代码符合 MISRA C 可能会得不偿失。在这种情况下，应详细评估是否采用 MISRA C。

5.2 软件开发过程

MISRA C 旨在在流程化的软件开发过程框架内使用。虽然 MISRA C 可以单独使用，但在流程中使用它更有好处，可以确保正确执行其他活动，例如：

- 软件需求，包括任何安全需求，是完整的、明确的和正确的；
- 到达编码阶段的设计规范是正确的，与需求一致，并且不包含任何其他功能；
- 由编译器产生的对象模块的行为与设计一致；
- 对目标模块进行了单元测试或集成测试，以识别和消除错误。

在代码提交评审或单元测试之前，程序员应该使用 MISRA C。一个项目在开发周期后期才检查是否符合 MISRA C 规则会带来大量重新编码、重新评审、重新测试的时间浪费。因此软件开发过程需要尽早应用 MISRA C。

对与安全相关的需求讨论不在本文档的讨论范围之内。

5.2.1 MISRA C 的流程活动要求

为了使用 MISRA C，有必要开发和记录：

- 一份符合性矩阵，显示如何检查每个 MISRA C 指南的符合性；
- 对合理的不符合项的偏离情况进行授权和记录。

软件开发过程还应该记录为避免运行时错误所采取的措施，并证明。例如，它应该包括对以下证明和记录的过程描述：

- 运行环境为程序提供了足够的资源，特别是处理时间和堆栈空间；
- 运行时错误，例如算术溢出不会出现在程序中：例如，通过对输入参数进行范围检查。

5.2.2 MISRA C 的流程活动期望

略

5.3 合规性

为了确保代码符合所有的 MISRA C 规则，应该制作一个符合性矩阵。这个矩阵列出了每个指导方针，并指出如何检查。对于大多数规则来说，最简单、最可靠和最经济有效的检查方法是使用一个或多个静态分析工具、编译器或这些工具的组合。如果一条规则不能通过工具完全检查，那么就需要手工检查。

5.4 偏离

在某些情况下，可能有必要偏离本文件中给出的指导方针，例如访问固定地址的 I/O 引脚无法符合 MISRA C，因为它将一个整数转换为指针。

```
#define PORT (*(volatile unsigned char *)0x0002)

PORT = 0x10u;
```

偏离过程应该在软件开发过程中记录和审核，可以通过正式的授权来防止个别程序员偏离规则。

偏离的审核批准过程应当正式化，但 MISRA C 没有强加任何过程，因为不同的组织使用的方法是不同的，例如：

- 提出、审核和批准的物理方法；
- 在批准之前需要的审查和证明的程度，可能会根据问题的指导方针而有所不同。

每条 MISRA C 准则都有影响偏离应用方式的一种类别，此类别有两种：项目偏离和特定偏离。

当 MISRA C 准则在通用情况下发生偏离时，就使用项目偏离。例如，如果在通信驱动程序中使用了内存映射 I/O，那么在构成该驱动程序的所有文件中提出一个项目偏离来覆盖内存映射 I/O 的使用可能是合适的。项目偏离通常是在项目早期提出的，尽管不是唯一的。

当 MISRA C 准则对单个文件中的单个实例产生偏离时，将使用特定偏离。

偏离记录应包含以下内容：

- 偏离了哪条 MISRA C 准则；
- 允许偏离的情况：对于项目的偏差，这可能是一个通用的描述，如“所有代码在模块 X 中”，但对于特定的偏差，它将识别出不合规代码发生的位置；
- 对偏差进行论证，包括与其他可能性相比，偏差的风险评估；
- 说明如何确保安全，例如证明不安全的情况不会发生，以及需要进行的任何额外审查或测试；
- 偏差的潜在后果以及已经采取的任何缓解措施。

附录 I 提供了一个偏离记录示例。

5.5 合规声明

略

6 指南介绍

6.1 条款的类别

每条 MISRA C 准则都属于“规则”和“指示”之一。

“指示”是指不可能提供执行符合性检查所需的完整描述的指导方针。为了能够执行该项检查，需要额外的信息，例如可能在设计文件或需求规范中提供的信息。静态分析工具可能能够帮助检查对指示的遵从性，但是不同的工具可能会对不遵从项给出不同的解释。

“规则”是一个提供了对需求的完整描述的指导方针，可以在不需要任何其他信息的情况下检查源代码是否符合规则。特别地，静态分析工具应该能够检查符合 6.5 节中描述限制的规则。

第 7 部分包含所有的指示，第 8 部分包含所有的规则。

directive	指示	无法提供执行性检查
rule	规则	有明确的规则，可以执行源码检查

6.2 指南要求的类别

指南有以下三种要求：

Mandatory	强制项	必须满足
Required	要求项	在 5.4 允许偏差范围内，必须满足
Advisory	推荐项	最好的选择，应该满足

6.3 条款的构成

略

6.4 条款的冗余

略

6.5 规则的可判定性

略

6.6 分析范围

略

6.7 多组织项目

略

6.8 自动代码生成

略

6.9 条款的描述

略

7 指示

7.1 实现

指示 1.1（要求） 程序输出所依赖的任何实现定义的行为都应被记录和理解

Dir 1.1 Any implementation-defined behaviour on which the output of the program depends shall be documented and understood

对于 C90 和 C99，本文档的附录 G 列出了那些实现定义的行为：

- 是否被认为有可能导致意外的程序操作；
- 可能出现在一个项目中，即使它符合所有其他 MISRA C 指南。

所有这些程序输出所依赖的实现定义的行为都必须文档化和被开发者理解。

原因

知道哪些程序输出是必然而非偶然的是非常重要的。

下面列出了一些安全相关的嵌入式软件可能依赖的常见的实现定义的行为。

核心行为

大多数程序需要的基本的实现定义的行为，包括以下内容：

- 如何识别编译产生的诊断信息；
- 独立系统的 main 函数一般声明为 void main(void)；
- 标识符中有效字符的数量；
- 源字符集和执行字符集；
- 平台整数的大小；
- #include 的名称如何映射到文件名和位于主机文件系统中。

扩展

语言扩展通常在嵌入式系统中使用，以提供对外设的访问和将对象放置到带有特殊属性的内存区域，例如 EEPROM、RAM。一个符合标准的实现允许提供扩展，前提是它们不违背标准定义。

标准库

标准库实现的某些方面可能很重要：

- 使用某些标准库函数时分配给 `errno` 的值；
- 实现时钟和计时功能；
- 文件系统的特性。

应用程序二进制接口

某些情况下，必须在 C 代码中嵌入汇编指令；也有可能需要为不同编译器或不同代码语言提供接口代码。编译器的应用二进制接口（ABI）提供了执行这个任务所需的信息，包括一些实现定义的行为：

- 函数参数怎么进栈和传入寄存器？
- 函数怎么传递返回值？
- 哪些寄存器必须由函数保存？
- 具有自动存储生命期的对象怎么与堆栈关联的？
- 数据的字节对齐要求是是怎样的？
- 结构是如何布置的，位域是如何分配给存储单元的？

有些处理器有一个标准的 ABI，所有实现都会使用它。在没有标准 ABI 存在时，平台应提供它自己的 ABI。

整数除法

在 C90 中，有符号整数除法或余数运算，其中任意一个操作数都为负数，可以向下舍入或向零舍入。在 C99 中是向零舍入的。

浮点数实现

浮点类型的实现可能对程序行为产生重大影响，例如以下方面：

- 值的范围和存储它们的精度；
- 浮点运算后的舍入方向；
- 转换为较窄的浮点类型或整数类型时的舍入方向；
- 在存在下溢、溢出和 NaNs 时的行为；
- 发生域和范围错误时库函数的行为；

7.2 编译和链接

指示 2.1（要求）所有源文件在编译时都不能出现任何编译错误

Dir 2.1 **All source files shall compile without any compilation errors**

即使存在编译错误，符合规范的编译器也允许生成目标模块。然而，结果程序执行后可能会产生意想不到的行为。

7.3 需求追溯

指示 3.1（要求）所有代码应可追溯至文档要求

Dir 3.1 **All code shall be traceable to documented requirements**

将代码追溯到文档化需求的方法应由项目确定。实现可追溯性的一种方法是根据相应的设计文档来审查代码，而设计文档又根据需求进行了审查。

7.4 代码设计

指示 4.1（要求） 尽量减少运行错误

Dir 4.1 Run-time failures shall be minimized

C 语言提供的内置运行时检查非常有限。虽然这可生成紧凑和快速的可执行代码，但它将运行时检查的负担交给了程序员。为了达到预期的健壮性，程序员应仔细考虑在可能发生运行时错误的地方添加动态检查。

若可以证明表达式求值时发生运行时错误的可能性为 0，在这种情况下，不需要动态检查，只要论证被记录在案。任何这样的文件都应该包括论据所依赖的假设，这些信息可以在代码的后续修改中使用，以确保参数仍然有效。

所有被用于最小化运行时错误的技术都应该设计并形成文档，例如在设计标准、测试计划、静态分析配置文件和代码审查检查列表中，这些技术的性质可能取决于项目的完整性需求。详见 5.2。

以下是需要动态检查的一些方面：

- 算术错误 表达式上溢、下溢、除 0、位移导致符号位丢失或错误，在算术表达式中应仔细考虑值范围和计算顺序；
- 指针运算 动态指针必须检查有效性，指针不能越界；
- 数组边界错误 在使用数组下标索引数组之前，确保数组下标在数组大小的范围内；
- 函数参数 在将参数传递给标准库函数之前，应该检查参数的有效性；
- 解指针 在一个指针被使用前，必须通过运行时检查确保非 NULL；
- 动态内存 分配动态内存必须检查是否成功，以及适时释放；

```
float32_t f1 = 1E38f;
float32_t f2 = 10.0f;
float32_t f3 = 0.1f;
float32_t f4 = ( f1 * f2 ) * f3; /* (f1 * f2) will overflow */
float32_t f5 = f1 * ( f2 * f3 ); /* no overflow because (f2 * f3)
                                * is (approximately) 1 */

if ( ( f3 >= 0.0f ) && ( f3 <= 1.0f ) )
{
    /*
     * no overflow because f3 is known to be in range 0..1 so the
     * result of the multiplication will fit in type float32_t
     */
    f4 = f3 * 100.0f;
}
```

指示 4.2（推荐） 所有汇编代码实现应文档化

Dir 4.2 All usage of assembly language should be documented

指示 4.3（要求） 所有汇编代码应封装隔离

Dir 4.3 Assembly language shall be encapsulated and isolated

所使用的汇编代码应封装在一个汇编函数中，或者封装在一个 C 函数中，或者定义为一个宏。

```
#define NOP asm(" NOP")
```

指示 4.4（推荐） 代码不能被//或/*...*/注释

Dir 4.4 Sections of code should not be “commented out”

不需要编译的代码，应该使用条件编译指令（例如`#if #ifdef`加注释）。`/* */`是无法嵌套的，使用`//`和`/* */`组合注释代码有可能带来意想不到的错误。

指示 4.5（推荐） 在同一命名空间中的具有重叠可见性的标识符在排版上应没有歧义

Dir 4.5 Identifiers in the same name space with overlapping visibility should be typographically unambiguous

根据用于显示字符集的字体，某些符号可能会显示相同，即使字符是不同的。这可能会导致开发人员混淆。

以下标识符定义区分方法是不允许的：

- 相同字母的大写与小写互换；
- 下划线增减；
- 字母“O”和数字“0”互换；
- 字母“l”（i）、数字“1”、字母“l”（el）互换
- 字母“S”和数字“5”互换
- 字母“Z”和数字“2”互换
- 字母“n”和字母“h”互换
- 字母“B”和数字“8”互换
- 字母序列“rn”和字母“m”互换

```
int32_t    id1_a_b_c;
int32_t    id1_abc;      /* Non-compliant */

int32_t    id2_abc;
int32_t    id2_ABC;      /* Non-compliant */

int32_t    id3_a_bc;
int32_t    id3_ab_c;     /* Non-compliant */

int32_t    id4_I;
int32_t    id4_l;        /* Non-compliant */

int32_t    id5_Z;
int32_t    id5_2;        /* Non-compliant */

int32_t    id6_O;
int32_t    id6_0;        /* Non-compliant */

int32_t    id7_B;
int32_t    id7_8;        /* Non-compliant */

int32_t    id8_rn;
int32_t    id8_m;        /* Non-compliant */

int32_t    id9_rn;
struct
{
    int32_t id9_m;        /* Compliant */
};
```

指示 4.6（推荐） 应该使用能指示大小和符号的类型定义代替基本数值类型

Dir 4.6 typedefs that indicate size and signedness should be used in place of the basic numerical types

基本定义的数据类型不能直接使用，因为无法表征数据大小。在 C99 中，<stdint.h>定义了类型。例如，在 32 位 C90 实现平台中，可能是以下方式定义类型：

```
typedef signed   char    int8_t;
typedef signed   short   int16_t;
typedef signed   int     int32_t;
typedef signed   long    int64_t;

typedef unsigned char    uint8_t;
typedef unsigned short   uint16_t;
typedef unsigned int     uint32_t;
typedef unsigned long    uint64_t;

typedef          float    float32_t;
typedef          double   float64_t;
typedef long     double   float128_t;
```

如果抽象类型是根据特定长度类型定义的，那么这些抽象类型就没有必要，甚至不需要指定大小或符号。例如，下面的代码定义了一个抽象类型，以公斤为单位表示质量，但没有指出它的大小或符号。

```
typedef uint16_t mass_kg_t;
```

当与标准库或项目控制之外的代码交互时，最好不要应用此准则。

指示 4.7（要求） 如果函数返回故障信息，那么应该测试该故障信息

Dir 4.7 If a function returns error information, then that error information shall be tested

被设计返回故障信息的功能列表应由项目确定，函数返回的故障信息应以有意义的方式进行测试。

指示 4.8（推荐） 如果在调用单元中从未对结构或联合的指针进行解引用，那么对象的实现应该被隐藏

Dir 4.8 If a pointer to a structure or union is never dereferenced within a translation unit, then the implementation of the object should be hidden

如果从未对 struct 或 union 的指针解引用，则不需要该 struct 或 union 的实现细节，并且应该保护其内容不受无意中更改。

Example

```
/* Opaque.h */

#ifndef OPAQUE_H
#define OPAQUE_H

typedef struct OpaqueType *pOpaqueType;

#endif
```

```

/* Opaque.c */

#include "Opaque.h"

struct OpaqueType
{
    /* Object implementation */
};

/* UseOpaque.c */

#include "Opaque.h"

void f ( void )
{
    pOpaqueType pObj;

    pObj = GetObject ( );    /* Get a handle to an OpaqueType object */

    UseObject ( pObj );      /* Use it... */
}

```

指示 4.9（推荐） 在可以互换情形下，应优先使用函数而非宏

Dir 4.9 A function should be used in preference to a function-like macro where they are interchangeable

大多数情况下，应优先使用函数而非宏。函数可以在编译时检查参数类型以及优先计算形参，可以避免实参可能的副作用。当然，某些情况下使用宏是非常有用的。

从以下方面考虑函数与宏的选择：

- 函数参数和结果类型检查的好处；
- C99 中内联函数的可用性，不过请注意内联的有效性是由实现定义的；
- 代码大小与执行速度之间的平衡；
- 编译时间计算是否需要侧重：常量宏在编译时计算，优于函数调用时计算；
- 函数参数是否有效；
- 易于理解和易维护。

```

#define EVAL_BINOP( OP, L, R ) ( ( L ) OP ( R ) )

uint32_t x = EVAL_BINOP ( +, 1, 2 );

上例中，不能使用函数代替宏，因为有个 C 操作符作为参数。

#define DIV2(X) ( ( X ) / 2 )

void f ( void )
{
    static uint16_t x = DIV2 ( 10 );    /* Compliant - call not permitted */
    uint16_t y = DIV2 ( 10 );          /* Non-compliant - call permitted */
}

```

上例中，使用宏来初始化具有静态存储时间的对象是符合要求的，因为静态变量初始化是在编译时执行的，此时不允许函数调用。

指示 4.10（要求）应采取预防措施，以防止头文件的内容被包含不止一次

Dir 4.10 **Precautions shall be taken in order to prevent the contents of a header file being included more than once**

在一个复杂程序系统中，可能一个头文件通过嵌套引用不止一次。如果这种多重包含导致多个或相互冲突的定义，那么这可能导致未定义或错误的行为。

可以使用下面两种方式之一，避免头文件重复包含。

```
<start-of-file>
#if !defined ( identifier )
#define identifier
/* Contents of file */
#endif
<end-of-file>

<start-of-file>
#ifndef identifier
#define identifier
/* Contents of file */
#endif
<end-of-file>
```

指示 4.11（要求）应该检查传递给库函数的参数的有效性

Dir 4.11 **The validity of values passed to library functions shall be checked**

标准库中的许多函数没有检查传递给它们的参数的有效性，同样，一些第三方库的函数也没有检查传递给它们的参数的有效性。依赖编译器进行检测无法避免一些风险，程序员应设计一些恰当的参数检测方法。

有许多方法可以达到本条要求，包括以下方法：

- 在函数调用前检查传入参数值；
- 在自定义库函数内部检查函数参数值；
- 生成执行检查的函数的“包装”版本，然后调用；
- 静态地证明输入参数永远不能取无效值。

指示 4.12（要求）不允许使用动态分配内存

Dir 4.12 **Dynamic memory allocation shall not be used**

标准库函数的动态内存分配释放函数会产生一些未定义行为，具体在规则 21.3 有详述。任何其他动态内存分配系统都可能表现出与标准库类似的未定义行为，因此需检查第三方运行环境，以保证不会无意间使用动态内存。

如果决定使用动态内存，应小心确保软件以可预测的方式运行。例如，动态内存的可能有以下风险：

- 没有足够内存——应确保在分配失败后有安全的响应；
- 根据不同使用模式和产生的碎片程度，执行分配或重新分配所需的执行时间差异很大。

在下例中，第一个 **free** 可能是未定义行为，因为 **p** 指针有可能是 **NULL**（未分配成功）。另外在调用 **free** 之后，存储在指针中的值没有改变，但在某些平台上，它所指向的内存可能不再存在，复制该指针的行为可能导致内存异常。

```

#include <stdlib.h>

void f ( void )
{
    char      *p = ( char * ) malloc ( 10 );
    char      *q;

    free ( p );
    q = p;      /* Undefined behaviour - value of p is indeterminate */

    p = ( char * ) malloc ( 20 );
    free ( p );
    p = NULL;   /* Assigning NULL to freed pointer makes it determinate */
}

```

指示 4.13（推荐） 以适当的顺序调用为操作资源而设计的函数

Dir 4.13 Functions which are designed to provide operations on a resource should be called in an appropriate sequence

提供资源操作的一组函数通常有三种操作：分配资源（例如打开文件）、取消资源（例如关闭文件）、其他对资源的操作（例如读文件）。

应以合适的顺序使用资源函数组。

```

/* These functions are intended to be paired */
extern mutex_t mutex_lock ( void );
extern void    mutex_unlock ( mutex_t m );

extern int16_t x;

void f ( void )
{
    mutex_t m = mutex_lock ( );

    if ( x > 0 )
    {
        mutex_unlock ( m );
    }
    else
    {
        /* Mutex not unlocked on this path */
    }
}

```

8 规则

8.1 标准 C 环境

规则 1.1（要求） 程序不能包含违反标准 C 语法和约束的内容，并且不能超出编译软件的限制

Rule 1.1 The program shall contain no violations of the standard C syntax and constraints, and

shall not exceed the implementation's translation limits

规则 1.2（推荐） 不应该使用语言扩展

Rule 1.2 Language extensions should not be used

但一般来说，嵌入式系统都会使用扩展语言，这是必要的。

规则 1.3（要求） 不能出现未定义行为或未确定行为

Rule 1.3 There shall be no occurrence of undefined or critical unspecified behaviour

具体的未定义行为或未确定行为需要根据标准 C 文档定义确定，本文档附录 H 列出了所有未定义和关键的未指定行为，以及 MISRA C 指南，以防止这些行为的发生。

8.2 未使用代码

规则 2.1（要求） 项目中不能有无法到达的代码

Rule 2.1 A project shall not contain unreachable code

无法到达的代码，有可能是程序员设计错误导致的，不允许无法到达的代码，可以避免此类错误。

规则 2.2（要求） 不能有死代码

Rule 2.2 There shall be no dead code

死代码的存在可能表明程序的逻辑有错误。由于死代码可能被编译器删除，它的存在可能会引起混乱。

```
extern volatile uint16_t v;

extern char *p;

void f ( void )
{
    uint16_t x;

    ( void ) v;          /* Compliant      - v is accessed for its side effect
                        *                  and the cast to void is permitted
                        *                  by exception                               */
    ( int32_t ) v;       /* Non-compliant - the cast operator is dead */
    v >> 3;              /* Non-compliant - the >> operator is dead */
    x = 3;               /* Non-compliant - the = operator is dead */
                        *                  - x is not subsequently read */
    *p++;               /* Non-compliant - result of * operator is not used */
    ( *p )++;           /* Compliant      - *p is incremented */
}
```

规则 2.3（推荐） 项目不应包含未使用的类型声明

Rule 2.3 A project should not contain unused type declarations

如果声明了类型但没有使用，那么审阅者就不清楚该类型是冗余的还是错误地。

规则 2.4（推荐） 项目不应包含未使用的标记声明

Rule 2.4 A project should not contain unused tag declarations

如果声明了 tag 但没有使用，那么审阅者就不清楚该类型是冗余的还是错误地。

```
void unusedtag ( void )
{
    enum state { S_init, S_run, S_sleep };    /* Non-compliant */
}

typedef struct record_t                        /* Non-compliant */
{
    uint16_t key;
    uint16_t val;
} record1_t;

typedef struct                                /* Compliant */
{
    uint16_t key;
    uint16_t val;
} record2_t;
```

上例中，record_t 是不会使用的，所以不符合本要求。

规则 2.5（推荐） 项目不应包含未使用的宏定义

Rule 2.5 A project should not contain unused macro declarations

规则 2.6（推荐） 函数不应包含未使用的标号声明

Rule 2.6 A function should not contain unused label declarations

```
void unused_label ( void )
{
    int16_t x = 6;

    label1:                        /* Non-compliant */
        use_int16 ( x );
}
```

规则 2.7（推荐） 函数中不应有未使用的函数参数

Rule 2.7 There should be no unused parameters in functions

如果一个函数的形参未被使用，函数的实现可能与需求不匹配。这条规则强调了这种潜在的不匹配。

8.3 注释

规则 3.1（要求） 在一条注释中，/*和//不能连在一起使用

Rule 3.1 The character sequences /* and // shall not be used within a comment

在/*的注释中，出现一条以/*或//开头的注释，有可能出现因缺失的*/而导致有用代码被注释掉。

```

/* some comment, end comment marker accidentally omitted

<<New Page>>

Perform_Critical_Safety_Function( X );

/* this comment is non-compliant */

```

上例中，因无意中遗漏*/而导致函数被注释掉了。

```

x = y // /*
    + z
    // */
;

```

上例中//改变了整个程序。没有//时， $x=y$ ；但现在变成了 $x=y+z$ 。这可能会引起混乱。

规则 3.2（要求） 行拼接“\”不能使用“//”注释

Rule 3.2 Line-splicing shall not be used in // comments

一条//注释以行拼接“\”结尾的话，会将新行也变成注释，这可能导致有用代码被无意中注释掉。

```

extern bool_t b;

void f ( void )
{
    uint16_t x = 0; // comment \
    if ( b )
    {
        ++x;          /* This is always executed */
    }
}

```

上例中 if(b)被注释掉了。

8.4 字符集和词法约定

规则 4.1（要求） 禁止使用八进制和十六进制转义序列

Rule 4.1 Octal and hexadecimal escape sequences shall be terminated

In this example, each of the strings pointed to by s1, s2 and s3 is equivalent to “Ag”.

```

const char *s1 = "\x41g"; /* Non-compliant */ const
char *s2 = "\x41" "g"; /* Compliant - terminated by end of literal */ const
char *s3 = "\x41\x67"; /* Compliant - terminated by another escape */

int c1 = '\141t'; /* Non-compliant */
int c2 = '\141\t'; /* Compliant - terminated by another escape */

```

规则 4.2（推荐） 不应使用三联符序列

Rule 4.2 Trigraphs should not be used

三联符序列是指由两个问号和特定的第三个字符组成的序列，它们可能会与两个问号的其他用法产生意外的混淆。（例如“??-”表示波浪号“~”，“??)”表示“]”）

```

"(Date should be in the form ??-??-??)"

```

上句会被编译器理解成:

```
"(Date should be in the form ~~]"
```

8.5 标识符

规则 5.1（要求） 具有外部作用域的标识符应能明确区分

Rule 5.1 External identifiers shall be distinct

在 C99 中，最低要求是外部标识符的前 31 个字符是严格区分的。

```
/*      1234567890123456789012345678901***** Characters */
int32_t engine_exhaust_gas_temperature_raw;
int32_t engine_exhaust_gas_temperature_scaled; /* Non-compliant */

/*      1234567890123456789012345678901***** Characters */
int32_t engine_exhaust_gas_temp_raw;
int32_t engine_exhaust_gas_temp_scaled;      /* Compliant */
```

规则 5.2（要求） 在同一作用域和名称空间中声明的标识符应该是不同的

Rule 5.2 Identifiers declared in the same scope and name space shall be distinct

在 C99 中，内部作用域标识符是以前 63 字符区分的。

对于规则 5.1 和 5.2，两标识符以非重要字符区分时，行为是未定义的。

规则 5.3（要求） 在内部作用域中声明的标识符不应该覆盖在外部作用域中声明的标识符

Rule 5.3 An identifier declared in an inner scope shall not hide an identifier declared in an outer scope

Example

```
void fn1 ( void )
{
    int16_t i;          /* Declare an object "i" */

    {
        int16_t i;      /* Non-compliant - hides previous "i" */

        i = 3;          /* Could be confusing as to which "i" this refers */
    }
}

struct astruct
{
    int16_t m;
};
```

```

extern void g ( struct astruct *p );

int16_t xyz = 0;                                /* Declare an object "xyz" */

void fn2 ( struct astruct xyz )                  /* Non-compliant - outer "xyz" is
                                                * now hidden by parameter name */
{
    g ( &xyz );
}

uint16_t speed;

void fn3 ( void )
{
    typedef float32_t speed; /* Non-compliant - type hides object */
}

```

规则 5.4（要求） 宏定义应能明确区分

Rule 5.4 Macro identifiers shall be distinct

宏应与其他宏名或其他参数名相区分，并且一个宏内的参数之间互不相同。

```

/*      1234567890123456789012345678901*****      Characters */
#define engine_exhaust_gas_temperature_raw    egt_r
#define engine_exhaust_gas_temperature_scaled egt_s /* Non-compliant */

/*      1234567890123456789012345678901*****      Characters */
#define engine_exhaust_gas_temp_raw           egt_r
#define engine_exhaust_gas_temp_scaled        egt_s /* Compliant */

```

规则 5.5（要求） 其他标识符应能与宏明确区分

Rule 5.5 Identifiers shall be distinct from macro names

该规则要求预处理之前存在的宏名与预处理之后存在的标识符不同。

保持宏名和标识符的不同有助于避免开发人员的混淆。

```

#define Sum(x, y) ( ( x ) + ( y ) )

int16_t Sum;

```

上例就是不合适的。

规则 5.6（要求） Typedef 重定义应具有唯一标识符

Rule 5.6 A typedef name shall be a unique identifier

重用一個 typedef 名称作为另一个 typedef 名称，或者作为函数、对象或枚举常量的名称，可能会导致开发人员的混淆。

例外情形：

结构体、联合和枚举的标签可以与 typedef 别名相同。

```

typedef float mass;

void func1 ( void )
{
    float32_t mass = 0.0f;          /* Non-compliant - reuse */
}

typedef struct list
{
    struct list *next;
    uint16_t     element;
} list;                             /* Compliant - exception */

typedef struct
{
    struct chain
    {
        struct chain *list;
        uint16_t     element;
    } s1;
    uint16_t length;
} chain;                             /* Non-compliant - tag "chain" not
                                     * associated with typedef */

```

规则 5.7（要求） Tag 标记应具有唯一标识符

Rule 5.7 A tag name shall be a unique identifier

例外情形：

结构体、联合和枚举的标签可以与 typedef 别名相同。

规则 5.8（要求） 具有外部作用域的对象或函数应具有唯一标识符

Rule 5.8 Identifiers that define objects or functions with external linkage shall be unique

规则 5.9（推荐） 具有内部链接的对象或函数应具有唯一标识符

Rule 5.9 Identifiers that define objects or functions with internal linkage should be unique

8.6 类型

规则 6.1（要求） 位域只能用合适的类型声明

Rule 6.1 Bit-fields shall only be declared with an appropriate type

合适的位域类型是指：

C90: unsigned int 或 signed int;

C99: unsigned int 或 signed int、布尔、实现平台允许的其他整数类型；

规则 6.2（要求） 一个 bit 的位域不能是有符号类型

Rule 6.2 Single-bit named bit fields shall not be of a signed type

因符号位会占用一个 bit。

8.7 字面量和常量

规则 7.1（要求） 八进制禁止使用

Rule 7.1 Octal constants shall not be used

```
extern uint16_t code[ 10 ];

code[ 1 ] = 109; /* Compliant - decimal 109 */
code[ 2 ] = 100; /* Compliant - decimal 100 */
code[ 3 ] = 052; /* Non-Compliant - decimal 42 */
code[ 4 ] = 071; /* Non-Compliant - decimal 57 */
```

规则 7.2（要求） 所有无符号整型应以“u”或“U”后缀标识

Rule 7.2 A “u” or “U” suffix shall be applied to all integer constants that are represented in an unsigned type

规则 7.3（要求） 小写字母“l”不能用于后缀标识，只能使用“L”，表征 long

Rule 7.3 The lowercase character “l” shall not be used in a literal suffix

小写 l (el) 不容易与数字 1 区分。

规则 7.4（要求） 字符串字面量不能赋值给对象，除非该对象的类型是“指向 const char 的指针”

Rule 7.4 A string literal shall not be assigned to an object unless the object's type is “pointer to const-qualified char”

不允许修改字符串字面量。修改字符串字面量是未定义行为。


```

/* Non-compliant - s is not const-qualified */
char *s = "string";

/* Compliant - p is const-qualified; additional qualifiers are permitted */
const volatile char *p = "string";

extern void f1 ( char *s1 );

extern void f2 ( const char *s2 );

void g ( void )
{
    f1 ( "string" );          /* Non-compliant - parameter s1 is not      */
                             /* const-qualified                      */
    f2 ( "string" );          /* Compliant                          */
}

char *name1 ( void )
{
    return ( "MISRA" );      /* Non-compliant - return type is not */
                             /* const-qualified                      */
}

const char *name2 ( void )
{
    return ( "MISRA" );      /* Compliant                          */
}

```

8.8 声明和定义

规则 8.1（要求） 对象类型应显式定义

Rule 8.1 Types shall be explicitly specified

C90 标准允许在某些情况下省略类型，在这种情况下隐式指定 `int` 类型。省略显式类型可能会导致混淆。

规则 8.2（要求） 函数原型声明中参数应带类型及命名

Rule 8.2 Function types shall be in prototype form with named parameters

早期的 C 版本，没有提供一种机制来根据相应的形参检查实参的数量或类型。在 C90 标准允许在某些情况下省略类型。

函数实参和形参的数量及类型，函数的期望和实际返回类型之间的不匹配可能导致未定义的行为。

该规则还要求为声明中的所有参数指定名称。参数名可以提供有关函数接口的有用信息，声明和定义之间的不匹配可能表明编程错误。


```

/* Compliant */
extern int16_t func1 ( int16_t n );

/* Non-compliant - parameter name not specified */
extern void func2 ( int16_t );

/* Non-compliant - not in prototype form */
static int16_t func3 ( );

/* Compliant - prototype specifies 0 parameters */
static int16_t func4 ( void );

/* Compliant */
int16_t func1 ( int16_t n )
{
    return n;
}

/* Non-compliant - old style identifier and declaration list */
static int16_t func3 ( vec, n )
int16_t *vec;
int16_t n;
{
    return vec[ n - 1 ];
}

```

规则 8.3（要求） 对象或函数的所有声明与定义都应该使用相同的名称和类型限定符

Rule 8.3 All declarations of an object or function shall use the same names and type qualifiers

在函数原型中指定参数名可以检查函数定义与其声明的接口是否一致。

等价类型是可交换的，例如 `int`、`signed int`、`signed` 是等价的。

```

extern void f ( signed int );
void f ( int ); /* Compliant - Exception */
extern void g ( int * const );
void g ( int * ); /* Non-compliant - type qualifiers */

extern int16_t func ( int16_t num, int16_t den );

/* Non-compliant - parameter names do not match */
int16_t func ( int16_t den, int16_t num )
{
    return num / den;
}

```

规则 8.4（要求） 当定义了具有外部链接的对象或函数时，一个兼容的声明应该是可见的

Rule 8.4 A compatible declaration shall be visible when an object or function with external linkage is defined

如果定义对象或函数时对象或函数的声明是可见的，则编译器必须检查声明和定义是否兼容。使用外部链接实现对象和函数声明的推荐方法是在头文件中声明它们，然后将头文件包含在所有需要它们的

代码文件中，包括定义它们的代码文件。

规则 8.5（要求） 一个外部对象或函数只能声明在一个文件中，仅声明一次

Rule 8.5 An external object or function shall be declared once in one and only one file

通常，外部对象在头文件中的声明，然后将头文件包含在定义或使用标识符的任何代码单元中。这样可以确保声明和定义、以及使用该对象或函数单元中声明的一致性。

```
/* featureX.h */
extern int16_t a; /* Declare a */

/* file.c */
#include "featureX.h"

int16_t a = 0; /* Define a */
```

规则 8.6（要求） 具有外部连接的标识符应该只有一个定义

Rule 8.6 An identifier with external linkage shall have exactly one external definition

如果一个标识符存在多个定义（在不同的文件中），或者根本不存在定义，则该行为是未定义的。该规则不允许在不同文件中有多个定义，即使定义是相同的。如果声明不同，或者将标识符初始化为不同的值，这就是未定义的行为。

```
/* file4.c */
int16_t k; /* Tentative definition - becomes external */

/* file5.c */
int16_t k = 0; /* External definition */
```

规则 8.7（推荐） 如果函数和对象仅在一个单元中引用，则不应该使用外部链接来定义

Rule 8.7 Functions and objects should not be defined with external linkage if they are referenced in only one translation unit

通过给对象提供内部链接或不提供链接来限制对象的可见性，可以减少无意中访问该对象的机会。类似地，通过提供内部链接来减少函数的可见性，可以减少不经意调用函数的机会。

遵守这个规则还可以避免与在另一个代码单元或库中标识符混淆的可能性。

规则 8.8（要求） 所有具有内部链接的函数和对象的声明应使用“static”标识

Rule 8.8 The static storage class specifier shall be used in all declarations of objects and functions that have internal linkage

规则 8.9（推荐） 如果对象的访问只是在单一函数或块中，那么对象应该在该函数或块范围内定义

Rule 8.9 An object should be defined at block scope if its identifier only appears in a single function

在块范围内定义对象减少了对对象被无意访问的可能性，并明确了在其他地方不应该访问该对象的意图。例如，在块范围声明了静态存储时间的对象不能从块之外直接访问。

```
uint32_t count ( void )
{
    static uint32_t call_count = 0;

    ++call_count;
    return call_count;
}
```

规则 8.10（要求） 内联函数应该用静态存储类声明

Rule 8.10 An inline function shall be declared with the static storage class

如果内联函数是用外部链接声明的，但没有在同一个翻译单元中定义，那么行为是未定义的。

调用一个声明为外部链接的内联函数，可能调用函数的外部定义，也可能使用 inline 定义。虽然这不会影响被调用函数的行为，但它可能会影响执行时间，从而对实时程序产生影响。

规则 8.11（推荐） 当声明具有外部链接的数组时，应显式指定其大小

Rule 8.11 When an array with external linkage is declared, its size should be explicitly specified

虽然可以声明一个不完整类型的数组并访问它的元素，但当数组的大小显式确定时，这样做更安全。为每个声明提供大小信息可以利于检查它们的一致性，它还可以允许静态检查器执行一些数组边界分析，而不需要分析多个代码单元。

```
extern int32_t array1[ 10 ]; /* Compliant */
extern int32_t array2[ ]; /* Non-compliant */
```

规则 8.12（要求） 在枚举列表中，隐式指定的枚举常量的值必须是唯一的

Rule 8.12 Within an enumerator list, the value of an implicitly-specified enumeration constant shall be unique

隐式指定的枚举常量的值比其前身大 1。如果第一个枚举常量是隐式指定的，那么它的值为 0。

显式指定的枚举常数具有关联常数表达式的值。

如果隐式指定的常量和显式指定的常量在枚举列表中混合使用，则可能导致值重合。这种重合可能是无意的，并可能导致意想不到的行为。

此条规则要求任何值的重合都应该是显式定义的。

```
/* Non-compliant - yellow replicates implicit green */
enum colour { red = 3, blue, green, yellow = 5 };

/* Compliant */
enum colour { red = 3, blue, green = 5, yellow = 5 };
```

规则 8.13（推荐） 指针应该尽可能指向 `const` 限定类型指针

Rule 8.13 A pointer should point to a `const`-qualified type whenever possible

指针应该指向 `const` 限定类型指针，除非用于修改对象或者复制给另一个非 `const` 限定类型指针。

```
uint16_t f ( uint16_t *p )
{
    return *p;
}
```

上例是违规的，因为函数内部不对 `p` 指向的对象作修改，应定义为下式：

```
uint16_t g ( const uint16_t *p )
```

规则 8.14（要求） 禁止使用类型限制符 “`restrict`”

Rule 8.14 The `restrict` type qualifier shall not be used

“`restrict`” 是 C99 标准引入的，它只可以用于限定和约束指针，并表明指针是访问一个数据对象的唯一且初始的方式，即它告诉编译器，所有修改该指针所指向内存中内容的操作都必须通过该指针来修改，而不能通过其它途径（其它变量或指针）来修改；这样做的好处是，能帮助编译器进行更好的优化代码，生成更有效率的汇编代码。如 `int *restrict ptr`，`ptr` 指向的内存单元只能被 `ptr` 访问到，任何同样指向这个内存单元的其他指针都是未定义的，直白点就是无效指针。

如果谨慎使用，`restrict` 类型限定符可能会提高编译器生成代码的效率，它还可以改进静态分析。但是，要使用 `restrict` 类型限定符，程序员必须确保两个或多个指针操作的内存区域不会重叠。

如果使用了不正确的 `restrict`，编译器将产生不符合预期行为的代码，这是一个很大的风险。

8.9 初始化

规则 9.1（强制） 具有自动存储生命期的对象的值在设置值之前不能进行读取操作

Rule 9.1 The value of an object with automatic storage duration shall not be read before it has been set

根据 C 标准，具有静态存储时间的对象会自动初始化为零，除非显式初始化。具有自动存储时间的对象不会自动初始化，因此可能具有不确定的值。

```
void f ( bool_t b, uint16_t *p )
{
    if ( b )
    {
        *p = 3U;
    }
}
```

```

void g ( void )
{
    uint16_t u;

    f ( false, &u );

    if ( u == 3U )
    {
        /* Non-compliant - u has not been assigned a value */
    }
}

```

规则 9.2（要求） 聚合或联合的初始化应该用大括号括起来

Rule 9.2 The initializer for an aggregate or union shall be enclosed in braces

初始化时用大括号区分对象中的子结构，可以使代码更清晰，利于程序员搞清复杂结构的初始化值，例如多维度数组、嵌套的结构或联合等。

但以下情况是允许的：

1. 数组用字符串字面量初始化；
2. 结构或联合的自动变量，用等价结构或联合值初始化；
3. 用特定的初始化结构初始化一个对象的子结构；
4. {0}，可以用于设置所有值为 0，可以用来初始化子结构，不需要多余的大括号。

```

int16_t y[ 3 ][ 2 ] = { 1, 2, 0, 0, 5, 6 }; /* Non-compliant */
int16_t y[ 3 ][ 2 ] = { { 1, 2 }, { 0 }, { 5, 6 } }; /* Compliant */
int16_t y[ 3 ][ 2 ] = { { 1, 2 }, { 0, 0 }, { 5, 6 } }; /* Compliant */

```

上例 **wgq** 第一句没有用大括号区分数组的列，因此不符合本规则。

```

float32_t a[ 3 ][ 2 ] = { 0 }; /* Compliant */
float32_t a[ 3 ][ 2 ] = { { 0 }, { 0 }, { 0 } }; /* Compliant */
float32_t a[ 3 ][ 2 ] = { { 0.0f, 0.0f },
                          { 0.0f, 0.0f },
                          { 0.0f, 0.0f } };
/* Compliant */

```

规则 9.3（要求） 数组不应被部分初始化

Rule 9.3 Arrays shall not be partially initialized

如果数组中的某个元素被初始化，那么整个数组应全部显式初始化。

为数组的每个元素提供显式初始化可以清楚地表明每个元素都已经考虑过了。

但以下情况是允许的：

1. 用{0}用于设置数组所有值为 0；
2. 可以使用特定初始化程序初始化数组，例如执行稀疏初始化；
3. 数组用字符串字面量初始化，可以不需要初始化每个元素。

```

/* Compliant */
int32_t x[ 3 ] = { 0, 1, 2 };

/* Non-compliant - y[ 2 ] is implicitly initialized */
int32_t y[ 3 ] = { 0, 1 };

```



```

/* Non-compliant - t[ 0 ] and t[ 3 ] are implicitly initialized */
float32_t t[ 4 ] = { [ 1 ] = 1.0f, 2.0f };

/* Compliant - designated initializers for sparse matrix */
float32_t z[ 50 ] = { [ 1 ] = 1.0f, [ 25 ] = 2.0f };

float32_t arr[ 3 ][ 2 ] =
{
    { 0.0f, 0.0f },
    { PI / 4.0f, -PI / 4.0f },
    { 0 } /* initializes all elements of array subobject arr[ 2 ] */
};

char h[ 10 ] = "Hello"; /* Compliant by Exception 3 */

```

规则 9.4（要求） 对象的元素只能初始化一次，禁止多次初始化

Rule 9.4 An element of an object shall not be initialized more than once

```

/*
 * Similar behaviour using designated initializers
 * Compliant - a2 is -5, -4, -3, -2, -1
 */
int16_t a2[ 5 ] = { [ 0 ] = -5, [ 1 ] = -4, [ 2 ] = -3,
                  [ 3 ] = -2, [ 4 ] = -1 };

/*
 * Repeated designated initializer element values overwrite earlier ones
 * Non-compliant - a3 is -5, -4, -2, 0, -1
 */
int16_t a3[ 5 ] = { [ 0 ] = -5, [ 1 ] = -4, [ 2 ] = -3,
                  [ 2 ] = -2, [ 4 ] = -1 };

struct mystruct
{
    int32_t a;
    int32_t b;
    int32_t c;
    int32_t d;
};

/*
 * Required behaviour using positional initialization
 * Compliant - s1 is 100, -1, 42, 999
 */
struct mystruct s1 = { 100, -1, 42, 999 };

/*
 * Similar behaviour using designated initializers
 * Compliant - s2 is 100, -1, 42, 999
 */
struct mystruct s2 = { .a = 100, .b = -1, .c = 42, .d = 999 };

/*
 * Repeated designated initializer element values overwrite earlier ones
 * Non-compliant - s3 is 42, -1, 0, 999
 */
struct mystruct s3 = { .a = 100, .b = -1, .a = 42, .d = 999 };

```

规则 9.5（要求） 当指定的初始化器用于初始化一个数组对象时，数组的大小应明确指定

Rule 9.5 Where designated initializers are used to initialize an array object the size of the array shall be specified explicitly

如果未显式指定数组的大小，则由已初始化的任何元素的最大下标决定。当使用指定的初始化器时，可能不总是清楚是哪一个初始化式有最高的索引值，特别是当初始化式包含大量元素时。

为了使意图清晰，数组的大小应该被明确地声明。同时可以检查是否定义了边界外的元素。

```
/* Non-compliant - probably unintentional to have single element */
int a1[ ] = { [ 0 ] = 1 };

/* Compliant */
int a2[ 10 ] = { [ 0 ] = 1 };
```

8.10 本征类型模型

8.10.1 原理

本章定义的本征类型模型并限制 C 的类型系统，有以下目的：

1. 支持更强的类型检查系统；
2. 定义一个基本的隐式和显式转换使用规则；
3. 促进可移植代码的使用；
4. 处理在 ISO C 中发现的一些类型转换异常。

8.10.2 本征类型模型

详见附录 C 和附录 D。

8.10.3 复合操作符和复合表达式

通过限制可能应用于复合表达式的隐式和显式转换，可以避免附录 C 中提到的一些问题，包括以下：

1. 整数表达式计算类型的混淆，因为这取决于操作数整数提升后的类型，算术结果的类型取决于实现平台 int 的大小；
2. 程序员中普遍存在一种误解，认为执行计算的类型受到被赋值或强制转换的结果的类型的的影响。这种错误的期望可能会导致意想不到的结果，

本征类型模型对复合表达式进行了进一步的限制，详见下文。

复合运算符：

- 乘除运算（* / %）
- 加减运算（二元+ 二元-）
- 位运算（& | ^）
- 移位运算（<< >>）
- 三元条件运算（?:）的第二和第三操作数是复合表达式

复合表达式是指有复合运算符的非常量表达式。

注意：

- 复合赋值并非复合表达式；
- 带圆括号的复合表达式也是复合表达式；
- 常量表达式并非复合表达式。

规则 10.1（要求） 操作数不能是不合适的本征类型

Rule 10.1 Operands shall not be of an inappropriate essential type

下表是本征类型模型的操作数要求，数字表示不允许的原因，无数字时表示是允许的。

运算类型	操作数	算术运算的本征类别					
		布尔	字符	枚举	有符号整型	无符号整型	浮点数
[]	整数	3	4				1
+（一元）		3	4	5			
-（一元）		3	4	5		8	
+ - ++ --	任一	3		5			
* /	任一	3	4	5			
%	任一	3	4	5			1
< > <= >=	任一	3					
== !=	任一						
! &&	任一		2	2	2	2	2
<< >>	左	3	4	5、6	6		1
<< >>	右	3	4	7	7		1
~ & ^	任一	3	4	5、yt6	6		1
?:	第一个		2	2	2	2	2
?:	第二、三						

1. 不能使用浮点数作操作数的运算，使用本质上是浮动类型的表达式，违反了运算约束；
2. 当操作数被要求为布尔值时，就应使用本质上为布尔类型的表达式作为操作数；
3. 当操作数被要求为数值时，不应使用本质上是布尔类型的操作数；
4. 当操作数被要求为数值时，不应使用本质上是字符类型的操作数；字符数据的数值是由实现平台定义的；
5. 不应在算术运算中使用枚举类型，因为枚举类型是由实现定义的整数类型；因此，涉及 `enum` 对象的操作可能会产生一个非预期类型的结果；但注意匿名枚举定义的枚举常量本质是有符号整型；
6. 移位运算和位运算只能使用本质是无符号整型的操作数；使用本质是有符号型的数值是由实现平台定义的；
7. 移位运算的右操作数应该是本质是无符号整型，以确保不会出现负移位这种未定义行为；
8. 一元的负号不能使用本质是无符号整型的操作数，因为结果的符号是由实现平台的 `int` 大小决定的。

以下都是违规的示例，注释中注明了违反条目。

```

f32a & 2U          /* Rationale 1 - constraint violation */
f32a << 2          /* Rationale 1 - constraint violation */

cha && bla         /* Rationale 2 - char type used as a Boolean value */
ena ? a1 : a2      /* Rationale 2 - enum type used as a Boolean value */
s8a && bla         /* Rationale 2 - signed type used as a Boolean value */
u8a ? a1 : a2      /* Rationale 2 - unsigned type used as a Boolean value */
f32a && bla        /* Rationale 2 - floating type used as a Boolean value */

bla * blb          /* Rationale 3 - Boolean used as a numeric value */
bla > blb          /* Rationale 3 - Boolean used as a numeric value */

cha & chb          /* Rationale 4 - char type used as a numeric value */
cha << 1           /* Rationale 4 - char type used as a numeric value */

ena--             /* Rationale 5 - enum type used in arithmetic operation */
ena * a1          /* Rationale 5 - enum type used in arithmetic operation */

s8a & 2           /* Rationale 6 - bitwise operation on signed type */
50 << 3U          /* Rationale 6 - shift operation on signed type */

u8a << s8a        /* Rationale 7 - shift magnitude uses signed type */
u8a << -1         /* Rationale 7 - shift magnitude uses signed type */

-u8a             /* Rationale 8 - unary minus on unsigned type */

```

例外情形:

非负的有符号整型常量表达式可以作为移位运算的右操作数。

规则 10.2 (要求) 本质上是字符类型的表达式不得当使用加减运算

Rule 10.2 Expressions of essentially character type shall not be used inappropriately in addition and subtraction operations

字符加减运算的正确的使用方式是:

1. 二元“+”操作, 一个操作数应是本征字符类型, 另一个应是本征 **signed** 或 **unsigned** 整型, 运算结果是本征字符类型;

2. 二元“-”操作, 第一个操作数应是本征字符类型, 第二个应是本征字符类型、本征 **signed** 或 **unsigned** 整型; 如果两个操作数都是本征字符类型, 运算结果是标准类型 (通常是 **int**), 否则运算结果是本征字符类型。

本征字符类型 (字符数据) 的表达式不应进行算术使用, 因为数据不代表数值, 只有上面两种操作是合理的, 因为它们允许对字符数据进行合理的操作。例如以下作用:

- 两个本征字符类型间的减法, 可能用于 '0' 到 '9' 之间的数字与相应的序数值间的转换;
- 本征字符类型与本征 **unsigned** 类型间的加法, 可能用于 '0' 到 '9' 之间的数字与相应的序数值间的转换;
- 本征字符类型减去本征 **unsigned** 类型间的减法, 可能用于将一个字符从小写转换为大写;

The following examples are compliant:

```
'0' + u8a /* Convert u8a to digit */
s8a + '0' /* Convert s8a to digit */
cha - '0' /* Convert cha to ordinal */
'0' - s8a /* Convert -s8a to digit */
```

The following examples are non-compliant:

```
s16a - 'a'
'0' + f32a
cha + ':'
cha - ena
```

规则 10.3（要求） 表达式的值不能赋值给具有更窄的本征类型或不同的本征类型类别的对象

Rule 10.3 The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category

本条规则已包含以下操作：

1. 标准定义的所有赋值操作；
2. `switch` 语句的 `case` 标号中的常量表达式到控制表达式的类型的转换。

C 语言为程序员提供了相当大的自由度，并允许不同算术类型之间的自动赋值。但这些隐式转换会导致非预期结果，如值丢失、符号丢失或精度丢失。详见附录 C。MISRA C 本征类型模型定义的强类型，可以减少此类问题。

以下是合规的。

```
uint8_t u8a = 0; /* By exception */
bool_t flag = ( bool_t ) 0;
bool_t set = true; /* true is essentially Boolean */
bool_t get = ( u8b > u8c );

ena = A1;
s8a = K1; /* Constant value fits */
u8a = 2; /* By exception */
u8a = 2 * 24; /* By exception */
cha += 1; /* cha = cha + 1 assigns character to character */

pu8a = pu8b; /* Same essential type */
u8a = u8b + u8c + u8d; /* Same essential type */
u8a = ( uint8_t ) s8a; /* Cast gives same essential type */

u32a = u16a; /* Assignment to a wider essential type */
```

以下是不合规的，因为操作数有不同的本征类型。

```
uint8_t u8a = 1.0f; /* unsigned and floating */
bool_t bla = 0; /* boolean and signed */
cha = 7; /* character and signed */
u8a = 'a'; /* unsigned and character */
u8b = 1 - 2; /* unsigned and signed */
u8c += 'a'; /* u8c = u8c + 'a' assigns character to unsigned */ use_uint32
( s32a ); /* signed and unsigned */
```

以下是不合规的，因为赋值给更窄的本征类型。

```
s8a = K2; /* Constant value does not fit */
u16a = u32a; /* uint32_t to uint16_t */

use_uint16 ( u32a ); /* uint32_t to uint16_t */ /* uint8_t

foo1 ( uint16_t x )
{
    return x; /* uint16_t to uint8_t */
}
```

例外情形：

非负的有符号整型常量表达式可以赋值给一个本征无符号整型。

{0}可以用于初始化集合类型或联合体。

规则 10.4（要求） 执行常用算术运算的操作符的两个操作数都应具有相同的本征类型类别

Rule 10.4 Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category

常用算术运算是指所有除了移位、逻辑&&、逻辑||、逗号运算之外的二元运算，另外，三元(?:)运算的第二和第三操作数也应符合此条规则。

C 语言为程序员提供了相当大的自由度，并允许不同算术类型之间的自动赋值。但这些隐式转换会导致非预期结果，如值丢失、符号丢失或精度丢失。详见附录 C。MISRA C 本征类型模型定义的强类型的使用，可以使程序员限制隐式转换的使用，以使程序产生正确的结果。

例外情形：

以下字符转换方式是允许的：

1. 二元“+”和“+=”可以有一个操作数是本征字符 char 类型，另一个是本征 signed 类型或本征 unsigned 类型；

2. 二元“-”和“-=”的左操作数可以是本征字符 char 类型，右操作数是本征 signed 类型或本征 unsigned 类型。

```
enum enuma { A1, A2, A3 } ena;
enum enumb { B1, B2, B3 } enb;
```

The following are compliant as they have the same *essential type category*:

```
ena > A1
u8a + u16b
```

The following is compliant by exception 1:

```
cha += u8a
```

The following is non-compliant with this rule and also violates Rule 10.3:

```
s8a += u8a /* signed and unsigned */
```


The following are non-compliant:

```
u8b + 2      /* unsigned and signed      */
enb > A1     /* enum<enumb> and enum<enuma> */
ena == enb   /* enum<enuma> and enum<enumb> */
u8a += cha   /* unsigned and char      */
```

规则 10.5（推荐） 表达式的值不应该被强制转换为不合适的本征类型

Rule 10.5 The value of an expression should not be cast to an inappropriate essential type

下表列出了不允许的强制转换，第一列是转换的目标。

Essential type category	from					
to	Boolean	character	enum	signed	unsigned	floating
Boolean		Avoid	Avoid	Avoid	Avoid	Avoid
character	Avoid					Avoid
enum	Avoid	Avoid	Avoid*	Avoid	Avoid	Avoid
signed	Avoid					
unsigned	Avoid					
floating	Avoid	Avoid				

将 void 转为其他任何类型都是不允许的，这将可能产生未定义行为。

例外情形：

值为 0 或 1 的整型常量可以转换为布尔型。

```
( bool_t ) false      /* Compliant - C99 'false' is essentially Boolean */
( int32_t ) 3U        /* Compliant */
( bool_t ) 0          /* Compliant - by exception */
( bool_t ) 3U         /* Non-compliant */

( int32_t ) ena        /* Compliant */
( enum enuma ) 3       /* Non-compliant */
( char ) enc           /* Compliant */
```

规则 10.6（要求） 复合表达式的值不能赋给具有更宽的本征类型的对象

Rule 10.6 The value of a composite expression shall not be assigned to an object with wider essential type

The following are compliant:

```
u16c = u16a + u16b;      /* Same essential type */
u32a = ( uint32_t ) u16a + u16b; /* Cast causes addition in uint32_t */
```

The following are non-compliant:

```
u32a = u16a + u16b;      /* Implicit conversion on assignment */
use_uint32 ( u16a + u16b ); /* Implicit conversion of fn argument */
```

规则 10.7（要求） 如果一个复合表达式用作常用算术运算的一个操作数，那么另一个操作数不应具有更宽的本征类型

Rule 10.7 If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type

限制复合表达式上的隐式转换意味着复合表达式内的算术操作序列必须在完全相同的基本类型中执行。这减少了开发人员可能产生的困惑。

The following are compliant:

```
u32a * u16a + u16b          /* No composite conversion */
( u32a * u16a ) + u16b       /* No composite conversion */
u32a * ( ( uint32_t ) u16a + u16b ) /* Both operands of * have
                                * same essential type */
u32a += ( u32b + u16b )      /* No composite conversion */
```

The following are non-compliant:

```
u32a * ( u16a + u16b ) /* Implicit conversion of ( u16a + u16b ) */
u32a += ( u16a + u16b ) /* Implicit conversion of ( u16a + u16b ) */
```

规则 10.8（要求） 复合表达式的值不应被转换为不同的本征类型类别或更宽的本征类型

Rule 10.8 The value of a composite expression shall not be cast to a different essential type category or a wider essential type

复合表达式转换为更宽类型是不允许的，因为在不同的平台中，计算结果可能不同。如计算下列表达式：

```
( uint32_t ) ( u16a + u16b );
```

在一个 16 位平台中，加法运算是在 16 位大小运算的，结果可能出现溢出而翻转；但在一个 32 位平台中，加法运算是在 32 位大小运算的，可以保存 16 平台丢失的更高位的信息。

强制转换为相同符号的更窄的本征类型是允许的，因为任何平台都会产生相同的结果。

```
( uint16_t ) ( u32a + u32b ) /* Compliant */
( uint16_t ) ( s32a + s32b ) /* Non-compliant - different essential
                                * type category */
( uint16_t ) s32a           /* Compliant - s32a is not composite */
( uint32_t ) ( u16a + u16b ) /* Non-compliant - cast to wider
                                * essential type */
```

8.11 指针类型转换

标准 C 中允许的指针转换是：

- 指针类型转为 void；
- 指针类型转为运算类型；
- 运算类型转为指针类型；

- 指针类型转为另一指针类型；

尽管标准允许运算类型与指针类型间的转换，但实际上非整数运算类型与指针类型间的转换是未定义的，也就是只允许整型与指针间的转换。

以下允许的指针转换不需要显示转换：

- 指针类型转为布尔型（C99）；
- 空指针常量转为指针类型；
- 从指针类型到兼容指针类型的转换，前提是目标类型具有源类型的所有类型限定符；
- 指向对象或不完整类型的指针与 `void*` 或其限定版本之间的转换，前提是目标类型具有源类型的所有类型限定符。

在 C99 中，任何不属于这个指针转换子集的隐式转换都违反了约束；在 C90 中，任何不属于这个指针转换子集的隐式转换都属于未定义行为。

指针类型与整数类型之间的转换都是由实现平台定义的。

规则 11.1（要求） 函数指针与其他类型之间不能进行转换

Rule 11.1 Conversions shall not be performed between a pointer to a function and any other type

函数指针与其他类型之间的转换会导致未定义行为。

例外情形：

1. null 指针常量转为函数指针；
2. 函数指针转为 `void`；
3. 函数类型可以隐式转换为指向该函数类型的指针。

```
typedef void ( *fp16 ) ( int16_t n );
typedef void ( *fp32 ) ( int32_t n );

#include <stdlib.h>                                /* To obtain macro NULL */

fp16 fp1 = NULL;                                   /* Compliant - exception 1 */
fp32 fp2 = ( fp32 ) fp1;                          /* Non-compliant - function
                                                    * pointer into different
                                                    * function pointer */

if ( fp2 != NULL )                                /* Compliant - exception 1 */
{
}

fp16 fp3 = ( fp16 ) 0x8000;                        /* Non-compliant - integer into
                                                    * function pointer */
fp16 fp4 = ( fp16 ) 1.0e6F;                        /* Non-compliant - float into
                                                    * function pointer */

typedef fp16 ( *pfp16 ) ( void );

pfp16 pfp1;

( void ) ( *pfp1 ( ) ); /* Compliant - exception 2 - cast function
                        * pointer into void */
```

```
extern void f ( int16_t n );

f ( 1 );          /* Compliant - exception 3 - implicit conversion
                  * of f into pointer to function          */
fp16 fp5 = f;     /* Compliant - exception 3              */
```

规则 11.2（要求） 不能在指向不完整类型的指针和任何其他类型之间进行转换

Rule 11.2 Conversions shall not be performed between a pointer to an incomplete type and any other type

指向不完整类型的指针与其他类型指针之间互转，可能没有正确对齐，导致未定义行为。

例外情形：

1. Null 指针常量可以转为不完整类型；
2. 不完整类型转为 void。

```
struct s;          /* Incomplete type          */
struct t;          /* A different incomplete type          */
struct s *sp;
struct t *tp;
int16_t *ip;

#include <stdlib.h> /* To obtain macro NULL          */

ip = ( int16_t * ) sp; /* Non-compliant          */
sp = ( struct s * ) 1234; /* Non-compliant          */
tp = ( struct t * ) sp; /* Non-compliant - casting pointer into a
                        * different incomplete type          */
sp = NULL;            /* Compliant - exception 1          */

struct s *f ( void );

( void ) f ( );      /* Compliant - exception 2          */
```

规则 11.3（要求） 不能在指向不同对象类型的指针之间进行强制转换

Rule 11.3 A cast shall not be performed between a pointer to object type and a pointer to a different object type

在指向不同对象类型的指针之间互转，可能没有正确对齐，导致未定义行为。

例外情形：

允许将指向对象类型的指针转换为指向 char 类型、signed char 类型或 unsigned char 类型的指针。
标准保证指向这些类型的指针可以用于访问对象的单个字节。

```

uint8_t *p1;
uint32_t *p2;

/* Non-compliant - possible incompatible alignment */
p2 = ( uint32_t * ) p1;

extern uint32_t read_value ( void );
extern void print ( uint32_t n );

void f ( void )
{
    uint32_t u = read_value ( );
    uint16_t *hi_p = ( uint16_t * ) &u; /* Non-compliant even though
                                           * probably correctly aligned */

    *hi_p = 0; /* Attempt to clear high 16-bits on big-endian machine */
    print ( u ); /* Line above may appear not to have been performed */
}

```

以下示例是符合规则的，因为该规则适用于非限定的指针类型。它不阻止将类型限定符添加到对象类型。

```

const short *p;
const volatile short *q;

q = ( const volatile short * ) p; /* Compliant */

```

下面的例子是不合规的，因为非限定的指针类型是不同的，分别为"指向 `const int` 型的指针"和"指向 `int` 型的指针"。

```

int * const * pcpi;
const int * const * pcpci;

pcpci = ( const int * const * ) pcpi;

```

规则 11.4（推荐） 不应该在对象指针和整数类型之间进行转换

Rule 11.4 A conversion should not be performed between a pointer to object and an integer type

在对象指针和整数类型之间互转，可能没有正确对齐，导致未定义行为。

指针和整数类型之间的类型转换应该尽可能避免，但在寻址内存映射寄存器或其他硬件特定特性时可能是必要的。如果使用整数和指针之间的强制转换，应小心确保产生的任何指针不会导致 11.3 规则中讨论的未定义行为。

例外情形：

Null 指针常量可以转为对象指针。

```

uint8_t *PORTA = ( uint8_t * ) 0x0002; /* Non-compliant */

uint16_t *p;

int32_t addr = ( int32_t ) &p; /* Non-compliant */
uint8_t *q = ( uint8_t * ) addr; /* Non-compliant */
bool_t b = ( bool_t ) p; /* Non-compliant */

enum etag { A, B } e = ( enum etag ) p; /* Non-compliant */

```

规则 11.5（推荐） 不应该执行从 void 指针到对象指针的转换

Rule 11.5 A conversion should not be performed from pointer to void into pointer to object

从 void 指针到对象指针的转换，可能没有正确对齐，导致未定义行为。在可能的情况下应该避免，但可能是必要的，例如在处理内存分配函数时。如果使用从指向对象的指针到指向 void 的指针的转换，应小心确保产生的任何指针不会导致 11.3 规则中讨论的未定义行为。

例外情形：

Null 指针常量可以转为对象指针。

```
uint32_t *p32;
void      *p;
uint16_t *p16;

p  = p32;          /* Compliant - pointer to uint32_t into
                   * pointer to void */
p16 = p;           /* Non-compliant */
p  = ( void * ) p16; /* Compliant */
p32 = ( uint32_t * ) p; /* Non-compliant */
```

规则 11.6（要求） 不能在 void 指针和运算类型之间进行强制转换

Rule 11.6 A cast shall not be performed between pointer to void and an arithmetic type

将一个整数转为 void 指针，可能没有正确对齐，导致未定义行为。

将 void 指针转换为整数可能会产生不能用所选整数类型表示的值，从而导致未定义的行为。

在非整数运算类型与 void 指针之间互转，是未定义的。

例外情形：

值为 0 的 null 指针常量表达式可以转为 void 指针。

```
void      *p;
uint32_t u;

/* Non-compliant - implementation-defined */
p = ( void * ) 0x1234u;

/* Non-compliant - undefined */
p = ( void * ) 1024.0f;

/* Non-compliant - implementation-defined */
u = ( uint32_t ) p;
```

规则 11.7（要求） 对象指针和非整型运算类型之间不应进行强制转换

Rule 11.7 A cast shall not be performed between pointer to object and a non-integer arithmetic type

将一个非整型运算类型转为对象指针，可能没有正确对齐，导致未定义行为。

将对象指针转换为非整型运算类型可能会产生不能用所选类型表示的值，从而导致未定义的行为。

在浮点类型与对象指针之间互转，是未定义的。

```

int16_t *p;
float32_t f;

f = ( float32_t ) p;    /* Non-compliant */
p = ( int16_t * ) f;    /* Non-compliant */

```

规则 11.8（要求） 强制转换不应该从指针所指向的类型中移除任何 **const** 或 **volatile** 限定符

Rule 11.8 A cast shall not remove any **const** or **volatile** qualification from the type pointed to by a pointer

规则 11.9（要求） 宏 **NULL** 应该定义成唯一允许的整数空指针常量形式

Rule 11.9 The macro **NULL** shall be the only permitted form of integer null pointer constant

使用 **NULL** 而不是 0，可以清楚地表明要使用 **NULL** 指针常量。

```

#define MY_NULL_1 0
#define MY_NULL_2 ( void * ) 0

if ( p1 == MY_NULL_1 )    /* Non-compliant */
{
}
if ( p2 == MY_NULL_2 )    /* Compliant */
{
}

/* Could also be stdio.h, stdlib.h and others */
#include <stddef.h>

extern void f ( uint8_t *p );

/* Compliant for any conforming definition of NULL, such as:
 * 0
 * ( void * ) 0
 * ( ( 0 ) )
 * ( ( ( 1 - 1 ) ) )
 */
f ( NULL );

```

8.12 表达式

规则 12.1（推荐） 表达式中操作符的优先级应该明确

Rule 12.1 The precedence of operators within expressions should be made explicit

Description	Operator or Operand	Precedence
Primary	identifier, constant, string literal, (expression)	16 (high)
Postfix	[] () (function call) . -> ++ (post-increment) -- (post-decrement) () { } (C99: compound literal)	15
Unary	++ (pre-increment) -- (pre-decrement) & * + - ~ ! sizeof defined (preprocessor)	14
Cast	()	13
Multiplicative	* / %	12
Additive	+ -	11
Bitwise shift	<< >>	10
Relational	< > <= >=	9
Equality	== !=	8
Bitwise AND	&	7
Bitwise XOR	^	6
Bitwise OR		5
Logical AND	&&	4
Logical OR		3
Conditional	?:	2
Assignment	= *= /= %= += -= <<= >>= &= ^= =	1
Comma	,	0 (low)

操作符 `sizeof` 的操作数应括在圆括号内；

对于优先级在 2 到 12 之间的表达式，任何操作数都应该用圆括号括起来。

C 语言有大量的操作符，它们的相对优先级并不直观，这可能会导致缺乏经验的程序员犯错误。使用圆括号使操作符优先级显式化，消除了产生非预期结果的可能性，它还使程序员的原始意图清晰地呈现给代码的审阅者或维护者。

但太多的括号使程序难于阅读，所以需要在括号和优先级明确要求之间进行妥协。

```

a[ i ]->n;      /* Compliant - no need to write ( a[ i ] )->n          */
*p++;          /* Compliant - no need to write *( p++ )              */
sizeof x + y;   /* Non-compliant - write either sizeof ( x ) + y      */
                * or sizeof ( x + y )                                */

/* Compliant - no need to write f ( ( a + b ), c ) */
x = f ( a + b, c );

/* Non-compliant
 * Operands of conditional operator (precedence 2) are:
 * == precedence 8 needs parentheses
 * a precedence 16 does not need parentheses
 * - precedence 11 needs parentheses
 */
x = a == b ? a : a - b;

/* Compliant */
x = ( a == b ) ? a : ( a - b );

```



```

/* Compliant
 * Operands of << operator (precedence 10) are:
 *   a precedence 16 does not need parentheses
 *   ( E ) precedence 16 already parenthesized
 */
x = a << ( b + c );

/* Compliant
 * Operands of && operator (precedence 4) are:
 *   a precedence 16 does not need parentheses
 *   && precedence 4 does not need parentheses
 */
if ( a && b && c )
{
}

/* Compliant
 * Operands of && operator (precedence 4) are:
 *   defined(X) precedence 14 does not need parentheses
 *   (E) precedence 16 already parenthesized
 */
#if defined ( X ) && ( ( X + Y ) > Z )

/* Compliant
 * Operands of && operator (precedence 4) are:
 *   !defined ( X ) precedence 14 does not need parentheses
 *   defined ( Y ) precedence 14 does not need parentheses
 * Operand of ! operator (precedence 14) is:
 *   defined ( X ) precedence 14 does not need parentheses
 */
#if !defined ( X ) && defined ( Y )

```

规则 12.2（要求） 移位操作的右操作数应该处于 0 与比左操作数的本征类型宽度少 1 的范围内

Rule 12.2 The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand

如果移位运算的右操作数为负数，或者大于或等于左操作数的宽度，则该行为未定义。例如，如果左移或右移的左操作数是 16 位整数，那么要确保只移动了 0 到 15 范围内的数字。

有多种方法可以确保遵循这一规则。最简单的方法是将右操作数设置为常量（然后可以静态检查其值）。使用无符号整数类型将确保操作数是非负的，因此只需要检查上限(在运行时动态地或通过检查)，否则两个限制都需要检查。

```

u8a = u8a << 7;          /* Compliant */
u8a = u8a << 8;          /* Non-compliant */
u16a = ( uint16_t ) u8a << 9; /* Compliant */

```

下例中，1u 的本征类型是 unsigned char。

```

1u << 10u;                /* Non-compliant */
( uint16_t ) 1u << 10u;   /* Compliant */
1UL << 10u;               /* Compliant */

```

规则 12.3（推荐） 不应使用逗号运算符

Rule 12.3 The comma operator should not be used

使用逗号运算符通常不利于代码的可读性，通常可以通过其他方法达到同样的效果。

规则 12.4（推荐） 常量表达式的求值不应导致无符号整数的翻转

Rule 12.4 Evaluation of constant expressions should not lead to unsigned integer wrap-around

在 16 位平台中，下例就有可能导致整型翻转。

```
#define BASE 65024u

switch ( x )
{
    case BASE + 0u:
        f ( );
        break;
    case BASE + 1u:
        g ( );
        break;
    case BASE + 512u:    /* Non-compliant - wraps to 0 */
        h ( );
        break;
}

#define DELAY 10000u
#define WIDTH 60000u

void fixed_pulse ( void )
{
    uint16_t off_time16 = DELAY + WIDTH;    /* Non-compliant */
}
```

8.13 副作用

规则 13.1（要求） 初始化列表不应该包含持久的副作用

Rule 13.1 Initializer lists shall not contain persistent side effects

C90 将具有聚合类型的自动对象的初始化列表限制为只包含常量表达式。但是，C99 允许聚合类型的自动对象的初始化列表包含运行时计算的表达式，它还允许作为复合字面值。在对初始化列表中的表达式求值期间，副作用发生的顺序是未指定的，因此，如果这些副作用是持久的，则初始化行为是不可预测的。

```
volatile uint16_t v1;

void f ( void )
{
    /* Non-compliant - volatile access is persistent side effect */
    uint16_t a[ 2 ] = { v1, 0 };
}

void g ( uint16_t x, uint16_t y )
{
    /* Compliant - no side effects */
    uint16_t a[ 2 ] = { x + y, x - y };
}
```

```

uint16_t x = 0u;

extern void p ( uint16_t a[ 2 ] );

void h ( void )
{
    /* Non-compliant - two side effects */
    p ( ( uint16_t[ 2 ] ) { x++, x++ } );
}

```

规则 13.2（要求） 表达式的值及其持久的副作用，在所有允许的求值顺序下应相同

Rule 13.2 The value of an expression and its persistent side effects shall be the same under all permitted evaluation orders

标准为编译器求表达式时提供了相当大的灵活性，大多数操作符都可以按任意顺序计算其操作数，除了以下情形：

1. 逻辑&&的第二个操作数只有在第一个操作数不为0时才计算；
2. 逻辑||的第二个操作数只有在第一个操作数为0时才计算；
3. 三元条件运算?:的第一个操作数先运算，根据第一个操作数的结果才选择运行第二个或第三个操作数之一；
4. 逗号运算“,”先计算第一个操作数，然后计算第二个操作数。

在两个操作序列间或一个完整的表达式中，有以下要求：

1. 不能写一个对象超过两次；
2. 不能写和读一个对象，除非读取对象的值用于运算后再存回该对象中；
3. 具有 volatile 限定类型的写访问不能超过一个；
4. 具有 volatile 限定类型的读访问不能超过一个。

下例是不合规的，i 读和写了两次，其读写顺序未知。

```

#define COPY_ELEMENT ( index ) ( a[( index )] = b[( index )] )

COPY_ELEMENT ( i++ );

```

下例是不合规的，v1 和 v2 的读取顺序是未知的。

```

extern volatile uint16_t v1, v2;
uint16_t t;

t = v1 + v2;

```

下例是合规的。

```

extern volatile uint8_t PORT;

PORT = PORT & 0x80u;

```

下例是不合规的，i 的副作用顺序是未知的。

```

uint16_t i = 0;

/*
 * Unspecified whether this call is equivalent to:
 *     f ( 0, 0 )
 * or   f ( 0, 1 )
 */
f ( i++, i );

```

规则 13.3（推荐） 包含自增(++)或自减(--)操作符的完整表达式除了由自增或自减操作符引起的副作用外，不应该有其他潜在的副作用

Rule 13.3 A full expression containing an increment (++) or decrement (--) operator should have no other potential side effects other than that caused by the increment or decrement operator

规则 13.4（推荐） 不应该使用赋值操作符的结果

Rule 13.4 The result of an assignment operator should not be used

注意，单纯的一句赋值操作例如 `a = 1`，其实并没有使用该运算的结果，而是自动忽视了。

```

x = y;                                /* Compliant */
a[ x ] = a[ x = y ];                 /* Non-compliant - the value of x = y
                                     * is used */

/*
 * Non-compliant - value of bool_var = false is used but
 * bool_var == false was probably intended
 */
if ( bool_var = false )
{
}

/* Non-compliant - values of c = 0 and b = c = 0 are used */
a = b = c = 0;

/* Non-compliant even though bool_var = true isn't evaluated */
if ( ( 0u == 0u ) || ( bool_var = true ) )
{
}

/* Non-compliant - value of x = f() is used */
if ( ( x = f ( ) ) != 0 )
{
}

/* Non-compliant - value of b += c is used */
a[ b += c ] = a[ b ];

```

规则 13.5（要求） 逻辑操作符“&&”或“||”的右操作数不应包含持久的副作用

Rule 13.5 The right hand operand of a logical && or || operator shall not contain persistent side effects

逻辑操作符“&&”或“||”的右操作数的运行是有条件的，并不一定会执行。

```
volatile uint16_t v;
uint16_t x;

/* Non-compliant - access to volatile v is persistent */
if ( ( x == 0u ) || ( v == 1u ) )
{
}

/* Non-compliant if fp points to a function with persistent side effects */
( fp != NULL ) && ( *fp ) ( 0 );

/* Non-compliant - h ( ) has a persistent side effect */
if ( ishigh && ( a == h ( x ) ) )
{
}
```

规则 13.6（强制） sizeof 操作符的操作数不应该包含任何有潜在副作用的表达式

Rule 13.6 The operand of the sizeof operator shall not contain any expression which has potential side effects

Sizeof 的操作数可以是表达式，也可以是一种数据类型；如果是表达式，该表达式并不会在程序运行期间运行。

C90 标准规定 sizeof 操作数的表达式不会在程序运行期间运行；C99 中 sizeof 操作数的表达式通常不会在程序运行期间运行；但如果操作数是隐藏大小的数组，需要计算数组时则会计算表达式；如果不需要，表达式是否计算是未定义的。

```
volatile int32_t i;
int32_t j;
size_t s;

s = sizeof ( j );          /* Compliant */
s = sizeof ( j++ );        /* Non-compliant */
s = sizeof ( i );          /* Compliant - exception */
s = sizeof ( int32_t );    /* Compliant */

volatile uint32_t v;

void f ( int32_t n )
{
    size_t s;

    s = sizeof ( int32_t[ n ] );          /* Compliant */
    s = sizeof ( int32_t[ n++ ] );        /* Non-compliant */
    s = sizeof ( void ( *[ n ] ) ( int32_t a[ v ] ) ); /* Non-compliant */
}
```

8.14 控制表达式

规则 14.1（要求） 循环计数变量不能是本征浮点类型

Rule 14.1 A loop counter shall not have essentially floating type

规则 14.2（要求） for 循环应该是格式良好的

Rule 14.2 A for loop shall be well-formed

For 循环的三条子句：

第一句应该是空的，或给循环计数变量一个初值，或定义一个循环计数变量并赋初值（C99）。

第二句应该是没有持久副作用的表达式，且使用循环计数变量和可选循环标志，且不得使用在 for 循环体中修改过的任何其他对象。

第三句应该是只修改循环计数变量的表达式，且不得使用在 for 循环体中修改过的任何对象。

在第二个子句中使用的循环控制标志被定义为一个单一的标识符，表示一个对象，该对象本质上是布尔类型。

例外情形：

三子句都为空是允许的，for(;;)表示无限循环。

```
bool_t flag = false;

for ( int16_t i = 0; ( i < 5 ) && !flag; i++ )
{
    if ( C )
    {
        flag = true;                /* Compliant - allows early termination
                                     * of loop                               */
    }

    i = i + 3;                      /* Non-compliant - altering the loop
                                     * counter                               */
}
```

规则 14.3（要求） 控制表达式不能是不变的

Rule 14.3 Controlling expressions shall not be invariant

该规则的控制表达式是指 if、while、for、do-while、switch 的控制语句和三元条件?:运算的第一操作数。

如果控制表达式是一个不变值，这很可能是一个编程错误，因为会出现程序不可及代码，编译器会移除这部分内容。

例外情形：

1. 无限循环表达式中，控制表达式值为定值是允许的；
2. 循环 do-while 的控制表达式是一个值为 0 的布尔量是允许的。


```

s8a = ( u16a < 0u ) ? 0 : 1;    /* Non-compliant - u16a always >= 0 */

if ( u16a <= 0xffffu )
{
    /* Non-compliant - always true */
}

while ( true )
{
    /* Compliant by exception 1 */
}

do
{
    /* Compliant by exception 2 */
} while ( 0u == 1u );

if ( 2 > 3 )
{
    /* Non-compliant - always false */
}

for ( s8a = 0; s8a < 130; ++s8a )
{
    /* Non-compliant - always true */
}

if ( ( s8a < 10 ) && ( s8a > 20 ) )
{
    /* Non-compliant - always false */
}

if ( ( s8a < 10 ) || ( s8a > 5 ) )
{
    /* Non-compliant - always true */
}

while ( s8a > 10 )
{
    if ( s8a > 5 )
    {
        /* Non-compliant - s8a not volatile */
    }
}

const uint8_t numcyl = 4u;

/*
 * Non-compliant - compiler is permitted to assume that numcyl always
 * has value 4
 */
if ( numcyl == 4u )
{
}

const volatile uint8_t numcyl_cal = 4u;

/*
 * Compliant - compiler assumes numcyl_cal may be changed by
 * an external method, e.g. automotive calibration tool, even
 * though the program cannot modify its value
 */
if ( numcyl_cal == 4u )
{
}

```

规则 14.4（要求） if 语句的控制表达式和循环语句的控制表达式本质上必须是布尔类型

Rule 14.4 The controlling expression of an if statement and the controlling expression of an iteration-statement shall have essentially Boolean type

```
int32_t *p, *q;

while ( p )           /* Non-compliant - p is a pointer */
{
}

while ( q != NULL )   /* Compliant */
{
}

while ( true )        /* Compliant */
{
}

extern bool_t flag;

while ( flag )        /* Compliant */
{
}

int32_t i;

if ( i )              /* Non-compliant */
{
}

if ( i != 0 )         /* Compliant */
{
}
```

8.15 控制流

规则 15.1（推荐） 不应使用 goto 语句

Rule 15.1 The goto statement should not be used

任意使用 goto 可能导致程序结构混乱，难于理解。

当然，在某些情况下 goto 的使用是有利的，但应该在规则 15.2 和 15.3 的限制下使用。

规则 15.2（要求） goto 语句应该只跳转到稍后在同一函数中声明的标签

Rule 15.2 The goto statement shall jump to a label declared later in the same function

限制 goto 的使用，禁止“返回”跳转，将有助于最小化可视化代码的复杂性。

```

void f ( void )
{
    int32_t j = 0;

L1:
    ++j;

    if ( 10 == j )
    {
        goto L2;          /* Compliant */
    }

    goto L1;              /* Non-compliant */

L2:
    ++j;
}

```

规则 15.3（要求） 任何被 **goto** 语句引用的标签都应该声明在同一个块中，
或者在任一包含该 **goto** 语句的块中

Rule 15.3 Any label referenced by a goto statement shall be declared in the same block, or in any block enclosing the goto statement

防止块之间的跳转或跳转到嵌套块中，有助于最小化可视化代码的复杂性。

Switch 语句的每一个 **case** 都是一个块。

```

void f1 ( int32_t a )
{
    if ( a <= 0 )
    {
        goto L2;          /* Non-compliant */
    }

    goto L1;              /* Compliant */

    if ( a == 0 )
    {
        goto L1;          /* Compliant */
    }

    goto L2;              /* Non-compliant */

L1:
    if ( a > 0 )
    {
        L2:
            ;
    }
}

```

下例是不合规的。

```

switch ( x )
{
    case 0:
        if ( x == y )
        {
            goto L1;
        }
        break;
    case 1:
        y = x;
L1:
        ++x;
        break;
    default:
        break;
}

```

规则 15.4（推荐） 用于终止任何循环语句的 break 或 goto 语句不应超过一个

Rule 15.4 There should be no more than one break or goto statement used to terminate any iteration statement

限制循环中的退出出口数量有助于最小化可视化代码的复杂性。在循环中使用一个 break 或 goto 语句可以提供一个中止循环的第二出口。

下例中出现多个退出点，是不合规的。

```

for ( x = 0; x < LIMIT; ++x )
{
    if ( BreakNow ( x ) )
    {
        break;
    }
    else if ( GotoNow ( x ) )
    {
        goto EXIT;
    }
    else
    {
        KeepGoing ( x );
    }
}

EXIT:
;

```

下例的内循环是合规的，但外循环有多个退出点，不合规。

```

while ( x != 0u )
{
    x = calc_new_x ( );

    if ( x == 1u )
    {
        break;
    }

    while ( y != 0u )
    {
        y = calc_new_y ( );

        if ( y == 1u )
        {
            goto L1;
        }
    }
}

L1:
z = x + y;

```

规则 15.5（推荐） 函数应该只有一个退出点

Rule 15.5 A function should have a single point of exit at the end

函数的 `return` 语句应是函数体最后一条语句；一个函数应只有一条 `return` 语句。

如果函数的退出点与产生持久副作用的语句穿插在一起，则很难确定在执行函数时将发生哪些副作用。

下例是不合规的。

```

bool_t f ( uint16_t n, char *p )
{
    if ( n > MAX )
    {
        return false;
    }

    if ( p == NULL )
    {
        return false;
    }

    return true;
}

```

规则 15.6（要求） 循环语句或选择语句的主体应该是复合语句

Rule 15.6 The body of an iteration-statement or a selection-statement shall be a compound-statement

复合语句是指用大括号组合的语句，单条语句也应使用大括号。

循环语句（`while`、`do-while`、`for`）和选择语句（`if`、`else`、`switch`）的主体应该是复合语句。

开发人员可能会错误地认为，由于语句的缩进，语句序列构成了循环语句或选择语句的主体。

在控制表达式之后偶然包含分号非常危险，这会导致空控制语句。

使用复合语句可以清楚地定义哪些语句实际上构成了主体，此外，缩进可能会导致开发人员将 `else` 语句与错误的 `if` 语句关联起来。

考虑下例，原始程序循环主体只有一条语句，需要维护，则可能出错。

```
while ( data_available )
    process_data ( );                /* Non-compliant */
```

新增的语句本应在循环主体中执行，但现在的做法无法达到要求。

```
while ( data_available )
    process_data ( );                /* Non-compliant */
    service_watchdog ( );
```

规则 15.7（要求） 所有 `if-else if` 结构应该用 `else` 语句结束

Rule 15.6 All `if - else if` constructs shall be terminated with an `else` statement

不论有没有可执行语句，最后的 `else` 语句都是必须的，以确保选择语句序列是正常结束的，该 `else` 语句最少应有一条执行语句或一条注释。这样利于代码审查。

```
if ( flag_1 )
{
    action_1 ( );
}
else if ( flag_2 )
{
    action_2 ( );
}
```

/ Non-compliant */*

The following shows a compliant terminating `else`.

```
else
{
    ;    /* No action required - ; is optional */
}
```

8.16 Switch 语句

规则 16.1（要求） 所有 `Switch` 语句应结构良好

Rule 16.1 All switch statements shall be well-formed

规则 16.2（要求） 在 `switch` 语句的主体是封闭最紧密的复合语句时才能使用 `switch` 标签

Rule 16.2 A switch label shall only be used when the most closely-enclosing compound statement is the body of a switch statement


```

switch ( x )
{
    case 1:          /* Compliant      */
        if ( flag )
        {
            case 2:      /* Non-compliant */
                x = 1;
            }
            break;
        default:
            break;
}

```

规则 16.3（要求） 每个 switch 子句应能被相应的 break 语句终止

Rule 16.3 An unconditional break statement shall terminate every switch-clause

如果开发者忘记在一条 case 语句块中用 break 结束，程序会继续向下一条 case 语句运行。这种做法虽然有时是故意的，但通常是错误的。

为确保这类错误能被发现，每个 switch 子句应该都用 break 语句终止。

在下例中，1 和 2 都是空语句，可以使用一条 break，但最好是每条都用一个 break，这样维护扩展时不会出错。

```

switch ( x )
{
    case 0:
        break;          /* Compliant - unconditional break      */
    case 1:              /* Compliant - empty fall through allows a group */
    case 2:
        break;          /* Compliant      */
    case 4:
        a = b;          /* Non-compliant - break omitted      */
    case 5:
        if ( a == b )
        {
            ++a;
            break;       /* Non-compliant - conditional break      */
        }
    default:
        ;               /* Non-compliant - default must also have a break */
}

```

规则 16.4（要求） 每个 switch 语句都应该有一个 default 标签

Rule 16.4 Every switch statement shall have a default label

注意，default 标签是防御性编程，default 标签应采取一些适当的操作。如果标签后面没有语句，则可以使用注释来解释为什么没有采取特定的操作。

```

switch ( x )
{
    case 0:
        ++x;
        break;
    case 1:
    case 2:
        break;
    default:          /* Compliant - default label is present          */
        errorflag = 1; /* should be non-empty if possible          */
        break;
}

enum Colours
{ RED, GREEN, BLUE } colour;

switch ( colour )
{
    case RED:
        next = GREEN;
        break;
    case GREEN:
        next = BLUE;
        break;
    case BLUE:
        next = RED;
        break;
}

/* Non-compliant - no default label.
 * Even though all values of the enumeration are
 * handled there is no guarantee that colour takes
 * one of those values
 */

```

规则 16.5（要求） default 标签应该作为 switch 语句的第一个或最后一个 switch 标号出现

Rule 16.5 A default label shall appear as either the first or the last switch label of a switch statement

规则 16.6（要求） 每个 switch 语句至少有两个 switch 子句

Rule 16.6 Every switch statement shall have at least two switch-clauses

只有单路径的 switch 语句是多余的，应该是一个编程错误。

```

switch ( x )
{
    default: /* Non-compliant - switch is redundant */
        x = 0;
        break;
}

switch ( y )
{
    case 1:
    default: /* Non-compliant - switch is redundant */
        y = 0;
        break;
}

```

```

switch ( z )
{
    case 1:
        z = 2;
        break;
    default: /* Compliant */
        z = 0;
        break;
}

```

规则 16.7（要求） switch 表达式不能是布尔类型

Rule 16.7 A switch-expression shall not have essentially Boolean type

如果是布尔型，应使用 if-else 语句。

8.17 函数

规则 17.1（要求） <stdarg.h>特性不能使用

Rule 17.1 The features of <stdarg.h> shall not be used

<stdarg.h>中有很多未定义行为，因此 va_list、va_arg、va_start、va_end 和 va_copy 都不能使用。

规则 17.2（要求） 函数不能直接或间接地调用自己

Rule 17.2 Functions shall not call themselves, either directly or indirectly

递归会带来超过可用堆栈空间的危险，这可能会导致严重的故障，除非递归被严格控制，否则在执行之前不可能确定最坏的情况是什么。

规则 17.3（强制） 函数不能隐式声明

Rule 17.3 A function shall not be declared implicitly

如果函数调用是在原型存在的情况下进行的，编译时可确保实参的数量与形参的数量匹配，并且每个实参都可以赋值给相应的形参。

如果函数是隐式声明的，C90 编译器将假定函数的返回类型为 int。因为隐式声明不提供原型，所以编译器不会知道函数形参的数量及其类型。不适当的类型转换可能导致传递实参和赋值返回值，以及其他未定义的行为。

If the function power is declared as:

```
extern double power ( double d, int n );
```

but the declaration is not visible in the following code then undefined behaviour will occur.

```

void func ( void )
{
    /* Non-compliant - return type and both argument types incorrect */
    double sql = power ( 1, 2.0 );
}

```

规则 17.4（强制） 所有非 void 返回类型的函数都必须有显式带表达式的 return 语句

Rule 17.4 All exit paths from a function with non-void return type shall have an explicit return statement with an expression

如果一个非 void 函数却没有返回值，但被调用使用了，该行为是未定义的。

```
uint16_t lookup ( uint16_t v )
{
    if ( ( v < V_MIN ) || ( v > V_MAX ) )
    {
        /* Non-compliant - no value returned. Constraint in C99 */
        return;
    }

    return table[ v ];
}
```

规则 17.5（推荐） 声明为数组类型的形参对应的函数实参必须具有适当数量的元素

Rule 17.5 The function argument corresponding to a parameter declared to have an array type shall have an appropriate number of elements

如果将形参声明为具有指定大小的数组，则每个函数调用中对应的实参应指向一个对象，该对象至少拥有与数组相同的元素。

对函数参数使用数组声明符比使用指针更清楚地指定函数接口，函数所期望的最小元素数是显式声明的，而这在指针中是不可能的。没有指定大小的函数形参数组声明符表示函数可以处理任意大小的数组。

推荐使用数组边界，因为它允许在函数体中实现越界检查，并对参数传递进行额外检查。在 C 语言中，将不正确大小的数组传递给指定大小的参数是合法的，但可能会导致意外的行为。

```

/*
 * Intent is that function does not access outside the range
 * array1[ 0 ] .. array1[ 3 ]
 */
void fn1 ( int32_t array1[ 4 ] );

/* Intent is that function handles arrays of any size */
void fn2 ( int32_t array2[ ] );

void fn ( int32_t *ptr )
{
    int32_t arr3[ 3 ] = { 1, 2, 3 };
    int32_t arr4[ 4 ] = { 0, 1, 2, 3 };

    /* Compliant - size of array matches the prototype */
    fn1 ( arr4 );

    /* Non-compliant - size of array does not match prototype */
    fn1 ( arr3 );

    /* Compliant only if ptr points to at least 4 elements */
    fn1 ( ptr );

    /* Compliant */
    fn2 ( arr4 );

    /* Compliant */
    fn2 ( ptr );
}

```

规则 17.6（强制） 数组形参的声明不能在 “[]” 内包含 static 关键字

Rule 17.6 The declaration of an array parameter shall not contain the static keyword between the []

C99 语言标准为程序员提供了一种机制来通知编译器一个数组形参包含指定的最小数量的元素。有些编译器能够利用这些信息为某些类型的处理器生成更有效的代码。

如果程序员所做的保证没有得到执行，并且元素的数量小于指定的最小值，那么行为是未定义的。

嵌入式系统使用的处理器不太可能提供利用程序员提供的额外信息所需的设施，程序不能满足保证的最小元素数量的风险超过了任何潜在的性能提高。

```

/* Non-compliant - uses static in array declarator */
uint16_t total ( uint16_t n, uint16_t a[ static 20 ] )
{
    uint16_t i;

    uint16_t sum = 0U;

    /* Undefined behaviour if a has fewer than 20 elements */
    for ( i = 0U; i < n; ++i )
    {
        sum = sum + a[ i ];
    }

    return sum;
}

```

```

extern uint16_t v1[ 10 ];
extern uint16_t v2[ 20 ];

void g ( void )
{
    uint16_t x;

    x = total ( 10U, v1 );    /* Undefined - v1 has 10 elements but needs
                               *                               at least 20 */
    x = total ( 20U, v2 );    /* Defined but non-compliant */
}

```

规则 17.7（要求） 具有非空返回类型的函数，其返回值应当被使用

Rule 17.7 The value returned by a function having non-void return type shall be used

如果调用函数而不使用返回值，有可能是一个错误。

如果不打算显式使用函数的返回值，则应该将其转换为 `void` 类型。

```

uint16_t func ( uint16_t para1 )
{
    return para1;
}

uint16_t x;

void discarded ( uint16_t para2 )
{
    func ( para2 );           /* Non-compliant - value discarded */
    ( void ) func ( para2 );  /* Compliant */
    x = func ( para2 );       /* Compliant */
}

```

规则 17.8（推荐） 函数参数使用时不应该进行写操作

Rule 17.8 A function parameter should not be modified

函数形参的行为与具有自动存储时间的对象相同。

虽然 C 语言允许修改参数，但这样的使用可能会让人感到困惑，并与程序员的期望发生冲突。将形参复制到自动对象再写自动对象可能会少一些混淆。对于现代编译器，这通常不会导致任何存储或执行时间损失。

不熟悉 C 语言但熟悉其他语言的程序员可能会写一个形参，认为该效果会影响函数调用的变量。

```

int16_t glob = 0;

void proc ( int16_t para )
{
    para = glob;              /* Non-compliant */
}

void f ( char *p, char *q )
{
    p = q;                    /* Non-compliant */
    *p = *q;                  /* Compliant */
}

```


8.18 指针和数组

规则 18.1（要求） 由指针算术运算产生的指针，应与原指针指向相同的数组

Rule 18.1 A pointer resulting from arithmetic on a pointer operand shall address an element of the same array as that pointer operand

创建指向数组末尾以外的指针是标准定义的，并且是允许的。对数组末尾以外的指针进行解引用会导致未定义的行为，这是被本规则禁止的。

```
struct
{
    uint16_t x;
    uint16_t y;
    uint16_t z;
    uint16_t a[ 10 ];
} s;

uint16_t *p;

void f3 ( void )
{
    p = &s.x;
    ++p;          /* Compliant      - p points one beyond s.x          */
    p[ 0 ] = 1;    /* Non-compliant - undefined, dereference of address one
                    *                      beyond s.x which is not necessarily
                    *                      the same as s.y                */
    p[ 1 ] = 2;    /* Non-compliant - undefined          */

    p = &s.a[ 0 ]; /* Compliant      - p points into s.a          */
    p = p + 8;     /* Compliant      - p still points into s.a    */
    p = p + 3;     /* Non-compliant - undefined, p points more than one
                    *                      beyond s.a                */
}
```

规则 18.2（要求） 指针之间的减法只适用于指向同一数组元素的指针

Rule 18.2 Subtraction between pointers shall only be applied to pointers that address elements of the same array

如果操作数不是指向同一数组，或者任一操作数超出了数组范围，该行为是未定义的。

规则 18.3（要求） 关系操作符 “>”、“>=”、“<”和“<=”不能应用于指针类型的对象，除非它们指向同一个对象

Rule 18.3 The relational operators >, >=, < and <= shall not be applied to objects of pointer type except where they point into the same object

如果操作数不是指向同一对象，该行为是未定义的。

```

struct limits
{
    int32_t lwb;
    int32_t upb;
};

void f2 ( void )
{
    struct limits limits_1 = { 2, 5 };
    struct limits limits_2 = { 10, 5 };

    if ( &limits_1.lwb <= &limits_1.upb )    /* Compliant */
    {
    }
    if ( &limits_1.lwb > &limits_2.upb )    /* Non-Compliant */
    {
    }
}

```

规则 18.4（推荐） “+”、“-”、“+=”和“-=”操作符不应应用于指针类型的表达式

Rule 18.4 The +, -, += and -= operators should not be applied to an expression of pointer type

使用数组下标语法的数组索引 `ptr[expr]` 是指针算术的首选形式，因为它通常更清晰，因此比指针操作更不容易出错。任何显式计算的指针值都有可能访问意外的或无效的内存地址。数组索引也可以实现这种行为，但是下标语法可以简化手工检查。

新手可能会对 C 中的指针算术感到困惑。表达式 `ptr+1` 可能会被错误地解释为对 `ptr` 中保存的地址加 1，事实上，新的内存地址取决于指针目标的字节大小，如果不正确地应用 `sizeof`，这种误解可能导致意外的行为。

```

void fn1 ( void )
{
    uint8_t a[ 10 ];
    uint8_t *ptr;
    uint8_t index = 0U;

    index = index + 1U;    /* Compliant - rule only applies to pointers */

    a[ index ] = 0U;    /* Compliant */
    ptr = &a[ 5 ];    /* Compliant */

    ptr = a;
    ptr++;    /* Compliant - increment operator not + */
    *( ptr + 5 ) = 0U;    /* Non-compliant */
    ptr[ 5 ] = 0U;    /* Compliant */
}

```

规则 18.5（推荐） 不应使用超过两层的指针嵌套

Rule 18.5 Declarations should contain no more than two levels of pointer nesting

使用两层以上的指针嵌套会使代码行为难于理解，因此应该避免使用。

规则 18.6（要求） 自动存储类型对象的地址不能被复制给另一个在该自动存储类型对象消失后仍然存在的对象

Rule 18.6 The address of an object with automatic storage shall not be copied to another object that persists after the first object has ceased to exist

当对象的生存期到期时，对象的地址变得不确定。任何不确定地址的使用都会导致不确定的行为。

```
int8_t *func ( void )
{
    int8_t local_auto;

    return &local_auto;    /* Non-compliant - &local_auto is indeterminate
                           *                      when func returns */
}
```

规则 18.7（要求） 不应该声明未知大小的数组

Rule 18.7 Flexible array members shall not be declared

灵活数组成员的存在改变了 `sizeof` 操作符的行为，这可能是程序员无法预料的。将包含灵活数组成员的结构赋值给另一个相同类型的结构可能不会以预期的方式进行，因为它只复制那些灵活数组之前的元素。

```
#include <stdlib.h>

struct s
{
    uint16_t len;
    uint32_t data[ ];    /* Non-compliant - flexible array member */
} str;

struct s *copy ( struct s *s1 )
{
    struct s *s2;

    /* Omit malloc ( ) return check for brevity */
    s2 = malloc ( sizeof ( struct s ) + ( s1->len * sizeof ( uint32_t ) ) );

    *s2 = *s1;    /* Only copies s1->len */

    return s2;
}
```

规则 18.8（要求） 不能使用可变长度数组类型

Rule 18.8 Variable-length array types shall not be used

当块或函数原型中声明的数组的大小不是整数常量表达式时，会声明可变长度数组类型，它们通常作为存储在堆栈上的可变大小对象来实现，因此，它们的使用使得不可能静态地确定堆栈的大小。

如果变长数组的大小为负或为零，则行为未定义。

如果一个变长数组被使用在需要与另一种变长数组数组类型兼容的环境中，那么数组类型的大小应该是相同的，此外，所有大小都应为正整数。如果这些需求没有被满足，行为是未定义的。

如果在 `sizeof` 操作符的操作数中使用了变长数组类型，在某些情况下，数组大小表达式是否求值是

不确定的。

变长数组类型的每个实例在其生命周期开始时都有固定的大小，这会导致一些令人困惑的行为。例如：

```
void f ( void )
{
    uint16_t n = 5;

    typedef uint16_t Vector[ n ]; /* An array type with 5 elements */

    n = 7;

    Vector    a1;                  /* An array type with 5 elements */

    uint16_t a2[ n ];              /* An array type with 7 elements */
}

void f ( int16_t n )
{
    uint16_t vla[ n ];             /* Non-compliant - Undefined if n <= 0 */
}

void g ( void )
{
    f ( 0 );                       /* Undefined */
    f ( -1 );                      /* Undefined */
    f ( 10 );                     /* Defined */
}
```

8.19 重叠存储

规则 19.1（强制） 一个对象不能被赋值或复制给一个与该对象有重叠存储的对象

Rule 19.1 An object shall not be assigned or copied to an overlapping object

创建两个在内存中有重叠的对象，当一个对象被分配或复制给另一个对象时，行为是未定义的。

例外情形：

下列行为是定义良好的，因此是允许的：

1. 在两个完全重叠并具有兼容类型的对象之间赋值（忽略它们的类型限定符）；
2. 使用标准库函数 `memmove` 在部分或完全重叠的对象之间进行复制。

```

#include <string.h>

int16_t a[ 20 ];

void f ( void )
{
    memcpy ( &a[ 5 ], &a[ 4 ], 2u * sizeof ( a[ 0 ] ) ); /* Non-compliant */
}

void g ( void )
{
    int16_t *p = &a[ 0 ];
    int16_t *q = &a[ 0 ];

    *p = *q; /* Compliant - exception 1 */
}

```

规则 19.2（推荐） 关键字 union 不应使用

Rule 19.2 The union keyword should not be used

可以写入一个 union 成员，然后以一种良好定义的方式读取该 union 同一成员。然而，写 union 的一个成员，但读另一个成员，行为结果取决于成员间的相对大小：如果读的成员大小比写的成员宽，那么值是未确定的，其他由实现平台确定。

标准允许通过一个 unsigned char 数组类型的成员访问 union 成员的字节。但是，因为有可能访问至带有未指定值的字节，所以不应该使用联合。

如果不遵循本条规则，以下行为需要明确：

1. 填充——联合成员间插入的填充量有多少；
2. 对齐——联合内部任何结构的成员是如何对齐的；
3. 字节顺序——字的高字节存在最高低址还是最低地址？
4. 位序——位如何在字节内编号，位如何分配给位域。

```

uint32_t zext ( uint16_t s )
{
    union
    {
        uint32_t ul;
        uint16_t us;
    } tmp;

    tmp.us = s;
    return tmp.ul; /* unspecified value */
}

```

8.20 预处理指令

规则 20.1（推荐） #include 指令应该只在预处理指令或注释之后，其他代码之前

Rule 20.1 #include directives should only be preceded by preprocessor directives or comments

为了提高代码的可读性，代码文件中的所有**#include** 指令应该放在文件顶部。

另外，在包含标准库或头文件之前，使用相关函数会导致未定义行为。

```
/* f.h */
xyz = 0;

/* f.c */

int16_t
#include "f.h"    /* Non-compliant */

/* f1.c */
#define F1_MACRO

#include "f1.h"    /* Compliant */
#include "f2.h"    /* Compliant */

int32_t i = 0;

#include "f3.h"    /* Non-compliant */
```

规则 20.2（要求） '、"或\字符和/*或//字符序列不能出现在头文件名称中

Rule 20.2 The ', " or \ characters and the /* or // character sequences shall not occur in a header file name

头文件名中使用这些字符会导致未定义行为。

规则 20.3（要求） #include 指令后面应该有<filename>或“filename”序列

Rule 20.3 The #include directive shall be followed by either a <filename> or "filename" sequence

#include 指令后面除<filename>或“filename”之外，其他形式是未定义的。

规则 20.4（要求） 宏的定义不能与关键字同名

Rule 20.4 A macro shall not be defined with the same name as a keyword

使用宏来更改关键字的含义可能会令人困惑。如果包含了一个标准头文件，而定义了一个与关键字同名的宏，则该行为是未定义的。

规则 20.5（推荐） #undef 不应该使用

Rule 20.5 #undef should not be used

使用#undef 会使宏变得不清晰。

规则 20.6（要求） 看起来像预处理指令的标记不应该出现在宏参数中

Rule 20.6 Tokens that look like a preprocessing directive shall not occur within a macro argument

在宏参数中使用以预处理指令执行的标记，会导致未定义行为。


```

#define M( A ) printf ( #A )

#include <stdio.h>

void main ( void )
{
    M (
#ifdef SW          /* Non-compliant */
        "Message 1"
#else              /* Non-compliant */
        "Message 2"
#endif            /* Non-compliant */
    );
}

```

上例可以打印为#ifdef SW "Message 1" #else "Message 2" #endif 或者"Message 2"，或者其他未定义行为。

规则 20.7（要求） 由宏参数展开产生的表达式应该括在圆括号中

Rule 20.7 Expressions resulting from the expansion of macro parameters shall be enclosed in parentheses

如果宏参数没有使用圆括号，在参数展开时，可能会产生非预期行为。但如果宏参数不是作为表达式，则不必使用圆括号。

```

#define M1( x, y ) ( x * y )

r = M1 ( 1 + 2, 3 + 4 );

the macro expands to give:

r = ( 1 + 2 * 3 + 4 );

#define M3( x ) a ## x = ( x )

int16_t M3 ( 0 );

```

规则 20.8（要求） #if 或#elif 预处理指令的控制表达式应求值为 0 或 1

Rule 20.8 The controlling expression of a #if or #elif preprocessing directive shall evaluate to 0 or 1

强类型要求条件预处理指令的控制表达式应具有布尔值。

规则 20.9（要求） 所有在#if 或#elif 预处理指令的控制表达式中使用的标识符都必须在求值前使用#define 定义

Rule 20.9 All identifiers used in the controlling expression of #if or #elif preprocessing directives shall be #define' d before evaluation

规则 20.10（推荐） 不应该使用#和##预处理操作符

Rule 20.10 The # and ## preprocessor operators should not be used

使用#和##会使代码不清晰。

规则 20.11（要求） 紧跟在#操作符之后的宏参数不应该紧跟着一个##操作符

Rule 20.11 A macro parameter immediately following a # operator shall not immediately be followed by a ## operator

多个#、多个##或混合使用#和##预处理指令的求值顺序是未指定的。规则 20.10 不鼓励使用#和##。特别地，#操作符的结果是一个字符串字面量，将它粘贴到任何其他预处理标记都不太可能产生有效的标记。

```
#define A( x )      #x          /* Compliant */
#define B( x, y )   x ## y      /* Compliant */
#define C( x, y )   #x ## y     /* Non-compliant */
```

规则 20.12（要求） 作为#或##操作符操作数的宏参数，若其本身也需要进行进一步的宏替换，则它只能作为这些操作符的操作数使用

Rule 20.12 A macro parameter used as an operand to the # or ## operators, which is itself subject to further macro replacement, shall only be used as an operand to these operators

上下文中混合使用宏替换可能不会满足开发人员的期望。

在下例是不合规的，因为 x 将替换成 AA，但还会进一步宏替换，但 x 并不仅仅是##的参数。

```
#define AA          0xffff
#define BB( x )     ( x ) + wow ## x /* Non-compliant */

void f ( void )
{
    int32_t wowAA = 0;

    /* Expands as wowAA = ( 0xffff ) + wowAA; */
    wowAA = BB ( AA );
}
```

下例是合规的。

```
int32_t speed;
int32_t speed_scale;
int32_t scaled_speed;

#define SCALE( X ) ( ( X ) * X ## _scale )

/* expands to scaled_speed = ( ( speed ) * speed_scale ); */
scaled_speed = SCALE ( speed );
```

规则 20.13（要求） 第一个标记为#的行应该是一个有效的预处理指令

Rule 20.13 A line whose first token is # shall be a valid preprocessing directive

规则 20.14（要求） 所有的#else，#elif 和#endif 预处理器指令都应该与它们相关的#if， #ifdef 或#ifndef 指令在同一个文件中

Rule 20.14 All #else, #elif and #endif preprocessor directives shall reside in the same file as the #if, #ifdef or #ifndef directive to which they are related

若条件编译指令跨文件出现，会出现混乱。

8.21 标准库

规则 21.1（要求） **#define** 和**#undef** 不能用于保留标识符或保留宏名

Rule 21.1 `#define` and `#undef` shall not be used on a reserved identifier or reserved macro name

以下划线开头的标识符是保留标识符。

规则 21.2（要求） 不能声明保留的标识符或宏名

Rule 21.2 A reserved identifier or macro name shall not be declared

规则 21.3（要求） 不得使用<stdlib.h>的内存分配和释放函数

Rule 21.3 The memory allocation and deallocation functions of `<stdlib.h>` shall not be used

函数 `calloc`、`malloc`、`realloc` 和 `free` 不应使用。

这些动态内存分配函数可能会导致未定义行为。

规则 21.4（要求） 不得使用标准头文件<setjmp.h>

Rule 21.4 The standard header file `<setjmp.h>` shall not be used

指令 `setjmp` 和 `longjmp` 允许跳过正常的函数调度机制，有可能导致未定义或非预期行为。

规则 21.5（要求） 不得使用标准头文件<signal.h>

Rule 21.5 The standard header file `<signal.h>` shall not be used

信号处理包含实现平台定义的行为和未定义的行为。

规则 21.6（要求） 不能使用标准库的输入/输出函数

Rule 21.6 The Standard Library input/output functions shall not be used

规则 21.7（要求） 不得使用<stdlib.h>的 `atof`、`atoi`、`atol` 和 `atoll` 函数

Rule 21.7 The `atof`, `atoi`, `atol` and `atoll` functions of `<stdlib.h>` shall not be used

规则 21.8（要求） 不得使用<stdlib.h>的 `abort`、`exit`、`getenv` 和 `system` 库函数

Rule 21.8 The library functions `abort`, `exit`, `getenv` and `system` of `<stdlib.h>` shall not be used

规则 21.9（要求） 不得使用<stdlib.h>的 `bsearch` 和 `qsort` 库函数

Rule 21.9 The library functions `bsearch` and `qsort` of `<stdlib.h>` shall not be used

规则 21.10（要求） 不能使用标准库的时间和日期函数

Rule 21.10 The Standard Library time and date functions shall not be used

规则 21.11（要求） 不得使用标准头文件<tgmath.h>

Rule 21.11 The standard header file <tgmath.h> shall not be used

规则 21.12（推荐） 不应使用<fenv.h>的异常处理特性

Rule 21.12 The exception handling features of <fenv.h> should not be used

8.22 资源

规则 22.1（要求） 所有通过标准库函数动态获取的资源都应该被显式释放

Rule 22.1 All resources obtained dynamically by means of Standard Library functions shall be explicitly released

规则 22.2（强制） 通过标准库函数分配的内存块只允许释放一次

Rule 22.2 A block of memory shall only be freed if it was allocated by means of a Standard Library function

规则 22.3（要求） 同一文件不能在不同的流上同时打开进行读写访问

Rule 22.3 The same file shall not be open for read and write access at the same time on different streams

规则 22.4（强制） 不能尝试写入以只读方式打开的流

Rule 22.4 There shall be no attempt to write to a stream which has been opened as read-only

规则 22.5（强制） 指向 FILE 对象的指针不能被解引用

Rule 22.5 There shall be no attempt to write to a stream which has been opened as read-only

规则 22.6（强制） 指向 FILE 对象的指针不能在相关流关闭后使用

Rule 22.6 The value of a pointer to a FILE shall not be used after the associated stream has been closed

附录 A 所有指示及规则索引

实现 (The implementation)

D1.1 要求 程序输出所依赖的任何实现定义的行为都应被文档记录和能被理解

编译链接 (Compilation and build)

D2.1 要求 所有源代码都不能出现任何编译错误

需求追溯 (Requirements traceability)

D3.1 要求 所有代码可追溯至文档要求

代码设计 (Code design)

D4.1 要求 尽量减少运行错误

D4.2 推荐 所有汇编代码实现应文档化

D4.3 要求 所有汇编代码应封装隔离

D4.4 推荐 代码不能被//或/*...*/注释

D4.5 推荐 在同一命名空间中的具有重叠可见性的标识符在排版上应没有歧义

D4.6 推荐 应该使用能指示大小和符号的类型定义代替基本数值类型

D4.7 要求 如果函数返回故障信息，那么应该测试该故障信息

D4.8 推荐 如果在调用单元中从未对结构或联合的指针进行解引用，那么对象的实现应该被隐藏

D4.9 推荐 在可以互换情形下，应优先使用函数而非宏

D4.10 要求 应采取预防措施，以防止头文件的内容被包含不止一次

D4.11 要求 应该检查传递给库函数的参数的有效性

D4.12 要求 不允许使用动态分配内存

D4.13 推荐 以适当的顺序调用为操作资源而设计的函数

标准 C 环境 (A standard C environment)

R1.1 要求 程序不能包含违反标准 C 语法和约束的内容，并且不能超出编译软件的限制

R1.2 推荐 不应该使用语言扩展

R1.3 要求 不能出现未定义行为或未确定行为

未使用代码 (Unused code)

- R2.1 要求 项目中不能有无法到达的代码
- R2.2 要求 不能有死代码
- R2.3 推荐 项目不应包含未使用的类型声明
- R2.4 推荐 项目不应包含未使用的标记声明
- R2.5 推荐 项目不应包含未使用的宏定义
- R2.6 推荐 函数不应包含未使用的标记声明
- R2.7 推荐 函数中不应有未使用的函数参数

注释 (Comments)

- R3.1 要求 在一条注释中，`/*`和`/**`不能连在一起使用
- R3.2 要求 行拼接“`\`”不能使用“`/**`”注释

字符集和词法约定 (Character sets and lexical conventions)

- R4.1 要求 禁止使用八进制和十六进制转义序列
- R4.2 推荐 不应使用三联符序列

标识符 (Identifiers)

- R5.1 要求 具有外部作用域的标识符应能明确区分
- R5.2 要求 在同一作用域和名称空间中声明的标识符应该是不同的
- R5.3 要求 在内部作用域中声明的标识符不应该覆盖在外部作用域中声明的标识符
- R5.4 要求 宏标识符应能明确区分
- R5.5 要求 其他标识符应能与宏明确区分
- R5.6 要求 `Typedef` 重定义应具有唯一标识符
- R5.7 要求 `Tag` 标记应具有唯一标识符
- R5.8 要求 具有外部作用域的对象或函数应具有唯一标识符
- R5.9 推荐 具有内部链接的对象或函数应具有唯一标识符

类型 (Types)

- R6.1 要求 位域只能用合适的类型声明
- R6.2 要求 一个 `bit` 的位域不能是有符号类型

字面量和常量 (Literals and constants)

- R7.1 要求 八进制禁止使用
- R7.2 要求 所有无符号整型应以“`u`”或“`U`”后缀标识

- R7.3 要求 小写字母“l”不能用于后缀标识，只能使用“L”，表征 long
- R7.4 要求 字符串字面量不能赋值给对象，除非该对象的类型是“指向 const char 的指针”

声明和定义 (Declarations and definitions)

- R8.1 要求 对象类型应显式定义
- R8.2 要求 函数原型声明中参数应带类型及命名
- R8.3 要求 对象或函数的所有声明与定义都应该使用相同的名称和类型限定符
- R8.4 要求 当定义了具有外部链接的对象或函数时，一个兼容的声明应该是可见的
- R8.5 要求 一个外部对象或函数只能声明在一个文件中，仅声明一次
- R8.6 要求 具有外部连接的标识符应该只有一个定义
- R8.7 推荐 如果函数和对象仅在一个单元中引用，则不应该使用外部链接来定义
- R8.8 要求 所有具有内部链接的函数和对象的声明应使用“static”标识
- R8.9 推荐 如果对象的访问只是在单一函数或块中，那么对象应该在该函数或块范围内定义
- R8.10 要求 内联函数应该用静态存储类声明
- R8.11 推荐 当声明具有外部链接的数组时，应显式指定其大小
- R8.12 要求 在枚举列表中，隐式指定的枚举常量的值必须是唯一的
- R8.13 推荐 指针应该尽可能指向 const 限定类型指针
- R8.14 要求 禁止使用类型限制符“restrict”

初始化 (Initialization)

- R9.1 强制 具有自动存储生命期的对象的值在设置值之前不能进行读取操作
- R9.2 要求 聚合或联合的初始化应该用大括号括起来
- R9.3 要求 数组不应被部分初始化
- R9.4 要求 对象的元素只能初始化一次，禁止多次初始化
- R9.5 要求 当指定的初始化器用于初始化一个数组对象时，数组的大小应明确指定

本征类型模型 (The essential type model)

- R10.1 要求 操作数不能是不合适的本征类型
- R10.2 要求 本质上是字符类型的表达式不得不当地使用加减运算
- R10.3 要求 表达式的值不能赋值给具有更窄的本征类型或不同的本征类型类别的对象

- R10.4 要求 执行常用算术转换的操作符的两个操作数都应具有相同的本征类型类别
- R10.5 推荐 表达式的值不应该被强制转换为不合适的本征类型
- R10.6 要求 复合表达式的值不能赋给具有更宽的本征类型的对象
- R10.7 要求 如果一个复合表达式用作常用算术转换的一个操作数，那么另一个操作数不应具有更宽的本征类型
- R10.8 要求 复合表达式的值不应被转换为不同的本征类型类别或更宽的本征类型

指针类型转换 (Pointer type conversions)

- R11.1 要求 函数指针与其他类型间不能进行转换
- R11.2 要求 不能在指向不完整类型的指针和任何其他类型之间进行转换
- R11.3 要求 不能在指向不同对象类型的指针之间进行强制转换
- R11.4 推荐 不应该在对象指针和整数类型之间进行转换
- R11.5 推荐 不应该执行从 `void` 指针到对象指针的转换
- R11.6 要求 不能在 `void` 指针和运算类型之间进行强制转换
- R11.7 要求 对象指针和非整型运算类型之间不应进行强制转换
- R11.8 要求 强制转换不应该从指针所指向的类型中移除任何 `const` 或 `volatile` 限定符
- R11.9 要求 宏 `NULL` 应该定义成唯一允许的整数空指针常量形式

表达式 (Expressions)

- R12.1 推荐 表达式中操作符的优先级应该明确
- R12.2 要求 移位操作的右操作数应该处于 0 与比左操作数的本征类型宽度少 1 的范围内
- R12.3 推荐 不应使用逗号运算符
- R12.4 推荐 常量表达式的求值不应导致无符号整数的翻转

副作用 (Side effects)

- R13.1 要求 初始化列表不应该包含持久的副作用
- R13.2 要求 表达式的值及其持久的副作用，在所有允许的求值顺序下应相同
- R13.3 推荐 包含自增(`++`)或自减(`--`)操作符的完整表达式除了由自增或自减操作符引起的副作用外，不应该有其他潜在的副作用
- R13.4 推荐 不应该使用赋值操作符的结果
- R13.5 要求 逻辑操作符“`&&`”或“`||`”的右操作数不应包含持久的副作用
- R13.6 强制 `sizeof` 操作符的操作数不应该包含任何有潜在副作用的表达式

控制表达式 (Control statement expressions)

- R14.1 要求 循环计数变量不能是浮点型，或本质是浮点型
- R14.2 要求 for 循环应该是格式良好的
- R14.3 要求 控制表达式不能是不变的
- R14.4 要求 if 语句的控制表达式和循环语句的控制表达式本质上必须是布尔类型

控制流 (Control flow)

- R15.1 推荐 不应使用 goto 语句
- R15.2 要求 goto 语句应该只跳转到稍后在同一函数中声明的标签
- R15.3 要求 任何被 goto 语句引用的标签都应该声明在同一个块中，或者在任何包含该 goto 语句的块中
- R15.4 推荐 用于终止任何循环语句的 break 或 goto 语句不应超过一个
- R15.5 推荐 函数应该只有一个退出点
- R15.6 要求 循环语句或选择语句的主体应该是复合语句
- R15.7 要求 所有 if-else if 结构应该用 else 语句结束

Switch 语句 (Switch statements)

- R16.1 要求 所有 Switch 语句应结构良好
- R16.2 要求 在 switch 语句的主体是封闭最紧密的复合语句时才能使用 switch 标签
- R16.3 要求 每个 switch 子句应能被相应的 break 语句终止
- R16.4 要求 每个 switch 语句都应该有一个 default 标签
- R16.5 要求 default 标签应该作为 switch 语句的第一个或最后一个 switch 标号出现
- R16.6 要求 每个 switch 语句至少有两个 switch 子句
- R16.7 要求 switch 表达式不能是布尔类型

函数 (Functions)

- R17.1 要求 <stdarg.h>特性不能使用
- R17.2 要求 函数不能直接或间接地调用自己
- R17.3 强制 函数不能隐式声明
- R17.4 强制 所有非 void 返回类型的函数都必须有显式带表达式的 return 语句
- R17.5 推荐 声明为数组类型的形参对应的函数实参必须具有适当数量的元素
- R17.6 强制 数组形参的声明不能在 “[]” 内包含 static 关键字
- R17.7 要求 具有非空返回类型的函数，其返回值应当被使用

R17.8 推荐 函数参数使用时不应该进行写操作

指针和数组 (Pointers and arrays)

R18.1 要求 由指针算术运算产生的指针，应与原指针指向相同的数组

R18.2 要求 指针之间的减法只适用于指向同一数组元素的指针

R18.3 要求 关系操作符“>”、“>=”、“<”和“<=”不能应用于指针类型的对象，除非它们指向同一个对象

R18.4 推荐 “+”、“-”、“+=”和“-=”操作符不应应用于指针类型的表达式

R18.5 推荐 不应使用超过两层的指针嵌套

R18.6 要求 自动存储类型对象的地址不能被复制给另一个在该自动存储类型对象消失后仍然存在的对象

R18.7 要求 不应该声明未知大小的数组

R18.8 要求 不能使用可变长度数组类型

重叠存储 (Overlapping storage)

R19.1 强制 一个对象不能被赋值或复制给一个与该对象有重叠存储的对象

R19.2 推荐 关键字 union 不应使用

预处理指令 (Preprocessing directives)

R20.1 推荐 #include 指令应该只在预处理指令或注释之后，其他代码之前

R20.2 要求 '、"或\字符和/*或//字符序列不能出现在头文件名称中

R20.3 要求 #include 指令后面应该有<filename>或“filename”序列

R20.4 要求 宏的定义不能与关键字同名

R20.5 推荐 #undef 不应该使用

R20.6 要求 看起来像预处理指令的标记不应该出现在宏参数中

R20.7 要求 由宏参数展开产生的表达式应该括在圆括号中

R20.8 要求 #if 或#elif 预处理指令的控制表达式应求值为 0 或 1

R20.9 要求 所有在#if 或#elif 预处理指令的控制表达式中使用的标识符都必须在求值前使用#define 定义

R20.10 推荐 不应该使用#和##预处理操作符

R20.11 要求 紧跟在#操作符之后的宏参数不应该紧跟着一个##操作符

R20.12 要求 作为#或##操作符操作数的宏参数，若其本身也需要进行进一步的宏替换，则它只能作为这些操作符的操作数使用

- R20.13 要求 第一个标记为**#**的行应该是一个有效的预处理指令
- R20.14 要求 所有的**#else**，**#elif** 和**#endif** 预处理器指令都应该与与它们相关的**#if**，**#ifdef** 或**#ifndef** 指令在同一个文件中

标准库 (Standard libraries)

- R21.1 要求 **#define** 和**#undef** 不能用于保留标识符或保留宏名
- R21.2 要求 不能声明保留的标识符或宏名
- R21.3 要求 不得使用<stdlib.h>的内存分配和释放函数
- R21.4 要求 不得使用标准头文件<setjmp.h>
- R21.5 要求 不得使用标准头文件<signal.h>
- R21.6 要求 不能使用标准库的输入/输出函数
- R21.7 要求 不得使用<stdlib.h>的 **atof**、**atoi**、**atol** 和 **atoll** 函数
- R21.8 要求 不得使用<stdlib.h>的 **abort**、**exit**、**getenv** 和 **system** 库函数
- R21.9 要求 不得使用<stdlib.h>的 **bsearch** 和 **qsort** 库函数
- R21.10 要求 不能使用标准库的时间和日期函数
- R21.11 要求 不得使用标准头文件<tgmath.h>
- R21.12 推荐 不应使用<fenv.h>的异常处理特性

资源 (Resources)

- R22.1 要求 所有通过标准库函数动态获取的资源都应该被显式释放
- R22.2 强制 通过标准库函数分配的内存块只允许释放一次
- R22.3 要求 同一文件不能在不同的流上同时打开进行读写访问
- R22.4 强制 不能尝试写入以只读方式打开的流
- R22.5 强制 指向 **FILE** 对象的指针不能被解引用
- R22.6 强制 指向 **FILE** 对象的指针不能在相关流关闭后使用

附录 B 规则指南结构

略

huanghao14

附录 C C 的类型安全问题

因标准 C 允许大范围使用隐式转化，所以一般认为标准 C 的类型安全较差。隐式转换具有一定的安全风险。

C.1 类型转换

C.1.1 隐式转换

在标准 C 中，混合类型的表达式，甚至相同类型的表达式都允许发生隐式转换。

隐式转换有三类：

1. **整数提升** 将一个整数提升为 `signed int` 或 `unsigned int` 类型会在以下情况发生：位域、`(un)signed char`、`(un)signed short` 和 `enum` 等类型在使用 C90 的 6.2.1.1 部分及 C99 的 6.3.1.1 部分定义的运算时。整数提升保留原始值，但符号可能会改变（例如 `unsigned char` 被转换为 `signed int`）

2. **平衡转换** 当不同类型的操作数与特定的操作符一起使用时，运算平衡转换会使操作数转换为某通用类型，详见 C90 的 6.2.1.5 部分及 C99 的 6.3.1.8 部分。

3. **赋值转换**

一条表达式语句可能 0-3 种隐式转换。

下表示例中，`si` 和 `ui` 表示 32 位有符号和无符号整型，`uc` 是 8 位无符号整型。

表达式	整数提升	平衡转换	赋值转换	注意
<code>si = si + si;</code>				
<code>si = uc + uc;</code>	Y			<code>uc</code> 整数提升 <code>signed int</code>
<code>ui = si + ui;</code>		Y		<code>si</code> 平衡转换 <code>unsigned int</code>
<code>ui = ui + uc;</code>	Y	Y		<code>uc</code> 整数提升 <code>signed int</code> ，平衡转换 <code>unsigned int</code>
<code>ui = si;</code>			Y	<code>si</code> 赋值转换 <code>unsigned int</code>
<code>ui = uc + uc;</code>	Y		Y	<code>uc</code> 整数提升 <code>signed int</code> ，表达式结果赋值转换 <code>unsigned int</code>
<code>si = si + ui;</code>		Y	Y	<code>si</code> 平衡转换 <code>unsigned int</code> ，表达式结果赋值转换 <code>signed int</code>
<code>si = ui + uc;</code>	Y	Y	Y	<code>uc</code> 整数提升 <code>signed int</code> ，平衡转换 <code>unsigned int</code> ，表达式结果赋值转换 <code>signed int</code>

C.1.2 显式转换

ISO C 不要求检查所有显式类型转换，允许在不兼容的类型之间进行转换。

C.1.3 转换的担忧

隐式或显式转化可能导致以下几个问题：

- 值丢失：例如转换为不能表示值大小的类型；

- 符号丢失：例如有符号整型向无符号整型转换；
- 精度丢失：例如浮点类型向整型转换；
- 比特位丢失：例如一个指针类型转为不同的指针类型，可能会出现未预期的存储比特位错误；

虽然许多类型转换是安全的，但唯一可以保证对所有数据值和所有可能的一致性实现都是安全的转换是以下几种：

- 整数类型向更宽相同符号的整数类型转换；
- 浮点型向更宽浮点型转换；

任何潜在危险的类型转换都应该通过显式的方式来标识。

C.2 开发人员导致的混乱

由于不同的硬件平台间基本类型字大小不同，发生隐式转换的上下文可能不同，这就要求开发人员为特定项目维护不同的模型。对于开发人员来说，使用错误的模型是相对容易的，特别是当他们在多个具有不同实现平台的项目上同时工作时。

C.2.1 在整数提升中进行类型扩展

计算整数表达式的类型取决于任何整数提升后操作数的类型。两个 8 位值相乘的结果总是至少是 16 位宽的；但只有当实现平台大小至少为 32 位时，两个 16 位值相乘才会得到 32 位的结果。永远不要依赖通过整数提升获得的类型扩展，因为相关的实现定义的行为可能会导致开发人员的困惑。

考虑以下示例：

```
uint16_t u16a = 40000;           // unsigned short or unsigned int ?
uint16_t u16b = 30000;           // unsigned short or unsigned int ?
uint32_t u32x;                   // unsigned int or unsigned long ?

u32x = u16a + u16b;               // u32x = 70000 or 4464 ?
```

预期结果可能是 70000，但 u32x 的值依赖于实现平台 int 的大小。平台是 32 位大小时，则 u32x 的值是 70000；如果是 16 位平台，则 u32x 的值是 4464（70000%65536）。

C.2.2 类型评估混乱

同样的错误也可能出现在这样一种情形：开发者认为计算的类型是否转换受到结果的影响。例如 C.2.1 的示例中，开发者误以为计算是在 32 位环境中计算的，因为想要的结果是 u32x。

以下示例同样存在问题。

```
u32a = (uint32_t)(u16a * u16b);    /* May not be a 32-bit operation */
f64a = u16a / u16b;                /* Not floating point division */
f32a = (float32_t)(u16a / u16b);   /* Not floating point division */
f64a = f32a + f32b;                /* May be a low precision operation */
f64a = (float64_t)(f32a + f32b);   /* May be a low precision operation */
```

C.2.3 算术运算时符号位改变

整数提升可能导致运算结果未必能达到开发者预期。例如表达式 10 - u16a 的结果取决于运行平台；假设 u16a 的值是 100，如果运行平台是 32 位大小，则表达式的结果是 -90；但如果运行平台是 16 位大小，则表达式的结果是 65446。

C.2.4 位运算时符号位改变

将按位运算符应用于小的无符号整型时产生的整数提升可能会导致混淆，例如一个 `unsigned char` 类型的取反运算会生成一个带负号的 `signed int` 值，该运算首先将操作数提升为 `int` 类型，然后进行取反运算，这样其他额外的高位会被取反运算影响。额外位的数量取决于操作平台的 `int` 大小，如果取反运算之后紧跟一个右移位运算，则结果是非预期的。

```
u8a = 0xff;

if ( ~u8a == 0x00U ) /* This test will always fail */
```

附录 D 本征类型 (Essential types)

ISO C 标准定义的 C 语言包含许多弱点和不一致之处，表达式的标准类型并不总是能够完全描述所表示的数据的本质。

例如：

- **整数提升** 整数类型行为不一致。作为整数提升的结果，本质上是无符号的表达式通常具有 signed int 的标准类型（例如 unsigned char + unsigned char）。
- **整型常量** 整型常量只有（signed/unsigned）int、long 和 long long 类型，这使得类型一致性的问题复杂化（例如当常量作为函数参数传递时）。
- **字符常量** 字符常量（例如 'x'）的标准类型被定义为 int（并非 char），这模糊了字符数据（即“字符”）和数字数据（即“数值”）之间的区别。
- **逻辑表达式** C90 未定义布尔型，C99 才定义了 _bool 表示布尔型，但为了向后兼容，逻辑操作数一直是 int 而非 _bool。
- **位域** 标准 C 未定义位域类型。ISO C 定义的位域的标准类型为 _Bool、signed int 或 unsigned int 之一。位域的符号位是由实现定义的。
- **枚举类型** 枚举变量的类型由实现定义。实现可以自由使用任何能够表示枚举的整数类型。如果枚举包含负数，则类型将是有符号整数类型。如果枚举仅由非负值组成，则类型可以是有符号整数类型或无符号整数类型。
- **枚举常量** 无论用于实现枚举的类型是什么，枚举常量都始终是 int 类型。

D.1 表达式的本征类型种类

为缓解以上问题，MISRA C:2012 引入“本征类型”概念。

表达式的本征类型种类是以下之一：

- 本征布尔类型（Boolean）
- 本征字符类型（character）
- 本征枚举类型（enum）
- 本征有符号整型（signed）
- 本征无符号整型（unsigned）
- 本征浮点类型（floating）

下表表明标准类型与本征类型间的对应关系，另外 C99 实现可提供扩展的整数类型，每种类型都要分配一个适合其级别和符号的位置（见 C99 6.3.1.1 部分）。

Essential type category					
Boolean	character	signed	unsigned	enum<i>	floating
_Bool	char	signed char signed short signed int signed long signed long long	unsigned char unsigned short unsigned int unsigned long unsigned long long	named enum	float double long double

当描述有符号类型和无符号类型时，等级的概念特别相关；signed/unsigned long long 共享“最高”

的等级，而 `signed/unsignedchar` 共享“最低”的等级。在 ISO C99 标准中，“等级”是一个仅适用于整数类型的术语。

下述章节将详细描述本征类型与标准类型间的差异。

D.2 字符量的本征类型

具有标准 `char` 类型(即单字节字符)的对象，本征类型是 `char`。

D.3 最低级别的有符号和无符号整型（STLR 和 UTLR）

本征类型模型引入了最低级别有符号整型（STLR）和最低级别无符号整型（UTLR）概念。这使得整数常量表达式和位域可以使用比 `int` 更低级别的类型来表示本征类型。对于整数常量表达式，这是一种方便，因为它避免了将表达式或其操作数强制转换为更低等级类型的操作；对于不允许位域的级别低于 `int` 类型的平台，它避免了在某些情况下强制转换位域字段。

1. STLR 是指能表示特定整数常量表达式的值或满足位域的最大值和最小值所需的最低等级有符号整数类型；

2. UTLR 是指能表示特定整数常量表达式的值或满足位域的最大值所需的最低等级无符号整数类型。

D.4 位域的本征类型

位域的本征类型由下列项决定：

1. 对于一个用本质是布尔类型实现的位字段，本征类型是布尔类型；
2. 对于一个用本质是有符号整型实现的位字段，本征类型是能表示该位域的 STLR；
3. 对于一个用本质是布尔类型实现的位字段，本征类型是能表示该位域的 UTLR；

D.5 枚举的本征类型

需要考虑两种不同类型的枚举：

1. 有名枚举类型是指具有标记(tag)或用于任何对象、函数或类型定义的枚举；
2. 匿名枚举类型是指不具有标记(tag)和未用于任何对象、函数或类型定义的枚举；匿名枚举类型通常用于定义一组相关或不相关的常量。

每个有名枚举类型可以与其他有名枚举类型明确区分，即使使用完全相同的标记和枚举常量在内部作用域中声明。在本文档中，有名枚举类型的每个实例都表示为 `enum<i>`，`i` 用于区分。

用 `typedef` 定义有名枚举类型的别名，并不是一个新类型。

匿名枚举类型的本征类型是其标准类型。

下例中每个都是有名枚举类型，且都有独立的本征类型：

```
enum ETAG { A, B, C };
typedef enum { A, B, C } ETYPE;
typedef enum ETAG { A, B, C } ETYPE;
enum { A, B, C } x;
```

下面的类型定义都指向相同的 `enum<i>`:

```
typedef enum      { A, B, C } ETYPE;
typedef ETYPE     FTYPE;
```

D.6 字面值常量的本征类型

C99 定义了以下形式的整型常量类型:

- 整数型常量
- 枚举常量
- 字符常量

■ 整数型常量

1. 若整数型常量的标准类型是 `signed int`, 其本征类型为 `STLR`;
2. 若整数型常量的标准类型是 `unsigned int`, 其本征类型为 `UTLR`;

■ 枚举常量

在 C 中, 不管枚举的实现类型是什么, 也不管用于定义其值的任何初始化式表达式的类型是什么, 枚举常量的标准类型总是 `int`。举例说明, 用无符号值(例如 `500U`)初始化一个枚举常量, 该枚举常量始终是 `signed int` 标准类型。

枚举常量的本征类型由以下方式确定:

1. 如果一个枚举定义了枚举类型名称, 则枚举常量的本征类型是 `enum<i>`;
 - 1.1 如果一个有名枚举类型用于定义一个本征布尔类型, 则枚举常量的本征类型是布尔;
2. 如果一个枚举是匿名枚举类型, 则每个枚举常量的本征类型是其值的 `STLR`。

下例中每个枚举常量及对象 `x`, 其本征类型都是 `enum<i>`;

```
enum ETAG { A = 8, B = 64, C = 128 } x;
```

下例匿名枚举类型定义了一系列常量。每个常量的本征类型是 `STLR`。因此在一个 8 位 `char` 和 16 位 `short` 的平台中, `A` 和 `B` 本征类型是 `signed char`, `C` 的本征类型是 `signed short`。

```
enum { A = 8, B = 64, C = 128 };
```

D.7 表达式的本征类型

本节中未列出的任何表达式的本征类型都与其标准类型相同。

■ 逗号运算 (,)

运算结果的本征类型就是右操作数的本征类型。

■ 关系运算 (<, <=, >=, >), 比较运算 (==, !=) 和逻辑运算 (&&, ||, !)

运算结果的本征类型是布尔。

■ 移位运算 (<<, >>)

1. 如果左操作数本质为无符号整型, 则为以下情形之一:
 - 1) 如果两个操作数都是整型常量, 则运算结果的本征类型是结果的 `UTLR` 型;
 - 2) 其他情况, 运算结果的本征类型是左操作数的本征类型;
2. 其他情况下为标准类型。

■ 按位取反 (~)

1. 如果操作数本质是无符号整型，则为以下情形之一：
 - 1) 如果操作数是整型常量，则运算结果的本征类型是结果的 UTLR 型；
 - 2) 其他情况，运算结果的本征类型是操作数的本征类型；
2. 其他情况下为标准类型。

■ 单目运算 (+)

如果操作数的本质是有符号或无符号整型，则运算结果的本征类型是操作数的本征类型；其他情况下为标准类型。

■ 单目运算 (-)

1. 如果操作本质是有符号整型，则为以下情形之一：
 - 1) 如果操作数是整型常量，则运算结果的本征类型是结果的 STLRL 型；
 - 2) 其他情况，运算结果的本征类型是操作数的本征类型；
2. 其他情况下为标准类型。

■ 条件运算 (?:)

1. 如果第二操作数及第三操作数有相同的本征类型，则运算结果的本征类型与此相同；
2. 如果第二操作数及第三操作数都是有符号整型，则运算结果的本征类型是操作数的较高等级本征类型；
3. 如果第二操作数及第三操作数都是无符号整型，则运算结果的本征类型是操作数的较高等级本征类型；
4. 其他情况下为标准类型。

■ 通用算术运算 (* / % + - & | ^)

1. 如果两操作数的本征类型都是有符号整型，则为以下情形之一：
 - 1) 如果表达式是整型常量表达式，则运算结果的本征类型是结果的 STLRL 型；
 - 2) 其他情况，运算结果的本征类型是操作数的较高等级本征类型；
2. 如果两操作数的本征类型都是无符号整型，则为以下情形之一：
 - 1) 如果表达式是整型常量表达式，则运算结果的本征类型是结果的 UTLRL 型；
 - 2) 其他情况，运算结果的本征类型是操作数的较高等级本征类型；
3. 其他情况为标准类型。

■ Char 的加法

如果一个操作数本征类型是 char，另一个操作数本征类型是有符号或无符号整型，则运算结果的本征类型是 char；其他情况下是标准类型。

■ Char 的减法

如果第一个操作数本征类型是 char，第二个操作数本征类型是有符号或无符号整型，则运算结果的本征类型是 char；其他情况下是标准类型。